```
// ==================== lib/types.ts ====================
/**
 * 离线资料库 —— 核心数据结构（接口冻结）
 *
 * WHY THIS FILE EXISTS FIRST
 *   老师对开题的批评里有一条是"全文检索技术路线无法核实"。
 *   核实不了的原因不是想法不行，而是**没有任何可运行的证据**。
 *
 *   所以这个模块的设计原则是：**纯 TypeScript，零依赖，可在 Node 里单测**。
 *   它同时满足三件事：
 *     1. 能在 PC 上跑单测 → 检索路线立刻变成"可核实"
 *     2. 能直接编译进 ArkTS 应用（无 any / 无动态导入 / 无平台 API）
 *     3. 软著登记需要的是"体现软件功能逻辑的源代码"，这本身就是
 *
 *   ⚠ 本文件**不导入任何 HarmonyOS Kit**。
 *      原因见 docs/11：HMS Kit 在缺少 HSP 的设备上会在**模块加载期**让整个应用崩溃。
 *      核心算法必须与平台解耦，才能既被单测、又不把应用拖死。
 */
/** 一份被导入的资料（一张照片、一个 PDF、一段文本）。 */
export interface DocItem {
  /** 稳定 id，入库后不变。 */
  id: string;
  /** 显示名（文件名或用户命名）。 */
  title: string;
  /** 来源类型。用于界面分组，也用于决定是否需要 OCR。 */
  source: DocSource;
  /** 原始内容的存储位置（URI 或沙箱内路径）。**不存内容本身**，避免重复占空间。 */
  uri: string;
  /** 入库时间（毫秒）。 */
  addedAt: number;
  /** 原文纯文本。OCR 之后填充；纯文本资料直接填充。 */
  text: string;
}
export type DocSource = 'camera' | 'album' | 'file' | 'note';
/**
 * 一个可检索的文本块。
 *
 * 为什么要分块：资料库的文档可能很长，检索需要定位到**具体片段**而不是整篇。
 * 分块策略直接影响召回质量，因此块大小是可调参数。
 */
export interface Chunk {
  /** 全局唯一：`${docId}#${seq}`。 */
  id: string;
  docId: string;
  /** 在原文中的序号。用于"前后文"与高亮定位。 */
  seq: number;
  /** 原文起点（字符偏移）。高亮回原图/原文时用。 */
  start: number;
  /** 原文终点（不含）。 */
  end: number;
```

```
  /** 块文本。 */
  text: string;
}
/**
 * 一条检索命中。
 *
 * 注意 `score` 与 `matched` 是两个不同的东西：
 *   score 是 BM25 相关度，用于排序；
 *   matched 是**具体命中了哪些词**，用于生成高亮与向用户解释"为什么给你这条"。
 */
export interface Hit {
  chunk: Chunk;
  /** BM25 得分。合并后的命中取被合并块中的**最高分**，不累加。 */
  score: number;
  /**
   * 命中的词项 → 该块中的出现次数。
   *
   * 用 `Map` 而不是 `Record<string, number>`：ArkTS 拒绝给索引签名类型赋对象字面量
   * （`arkts-no-untyped-obj-literals`），而 `Map` 在 TS 与 ArkTS 下行为一致。
   */
  matched: Map<string, number>;
  /** 高亮区间（相对 chunk.text 的字符偏移）。 */
  spans: Span[];
  /**
   * 这一条由几个原始块合并而成。未合并不设此字段（等价于 1）。
   *
   * 为什么需要暴露它：合并会改变"返回几条"的含义。
   * 界面上如果只说"找到 2 条"，用户不知道它原本是 5 个块，
   * 也就看不出合并起了多大作用。
   */
  mergedCount?: number;
}
/** 高亮区间，左闭右开。 */
export interface Span {
  start: number;
  end: number;
}
/** 检索选项。`topK` 由预算决策给出，不在这里写死。 */
export interface SearchOptions {
  /** 返回多少条。<=0 表示由得分分布自适应决定。 */
  topK: number;
  /** 最少返回几条（自适应时作为下界）。 */
  minK: number;
  /** 最多返回几条（自适应时作为上界）。 */
  maxK: number;
  /**
   * 是否把同一文档中**重叠或首尾相接**的命中块合并成一条。
   *
   * 默认关闭（不设即为不合并），这样既有的 87 个测试与评估基线不受影响。
   * 应用里打开它，因为对用户来说"同一段话被切成 3 条"是噪音。
```

```
   *
   * 合并发生在**选出 top-k 之后**：
   * 因此它会让最终显示条数变少，但不改变 k 的决策过程。
   * 这个顺序是刻意选的 —— 若在 k 之前合并，"按得分断层决定 k"的语义就变了。
   */
  mergeAdjacent?: boolean;
}
/** 检索结果，含决策依据，便于向用户解释。 */
export interface SearchResult {
  hits: Hit[];
  /** 实际使用的 k。 */
  k: number;
  /** k 是怎么定的：'fixed' | 'score-gap'。 */
  kSource: 'fixed' | 'score-gap';
  /** 用于决策的得分统计（自适应时为断层信息）。 */
  diagnostics: ScoreDiagnostics;
}
export interface ScoreDiagnostics {
  /** 命中的候选数（未截断前）。 */
  candidateCount: number;
  /** 最高分。 */
  topScore: number;
  /** 全部候选得分的均值和标准差。 */
  mean: number;
  std: number;
  /** 相邻得分之间的最大相对落差，以及它出现在第几位。 */
  maxGap: number;
  maxGapAt: number;
}
/** 预算决策的输入。字段都在手机上有真实含义，不是抽象参数。 */
export interface BudgetInput {
  /** 可用于检索的时间预算（毫秒）。 */
  latencyBudgetMs: number;
  /** 可用于本次检索的内存预算（KB）。 */
  memoryBudgetKb: number;
  /** 当前电量百分比（0-100）。低于阈值时降级。 */
  batteryPercent: number;
}
/** 预算决策的输出：这一次该取几条。 */
export interface BudgetDecision {
  k: number;
  /** 决策理由，直接显示给用户（"为什么只给你 3 条"）。 */
  reason: string;
}
/**
 * 持久化存档格式。
 *
 * 只存文档，不存倒排索引——理由见 `store.ts` 顶部的 DESIGN DECISION。
 * `version` 用于将来做格式迁移；目前只支持 1。
 */
```

```
export interface DocStoreSnapshot {
  version: number;
  docs: DocItem[];
}
// ==================== lib/tokenize.ts ====================
/**
 * 切词 —— 不依赖任何词典。
 *
 * WHY NO DICTIONARY
 *   手机上没有中文分词词典，也不该为一个个人资料库塞一个几十 MB 的词表。
 *   所以用**二元组（bigram）**：
 *     "离线资料库" → 离/线/资/料/库 + 离线/线资/资料/料库
 *
 *   这是中文检索里的经典做法：不用分词也能达到可用的召回率，
 *   代价是索引会大一些（约为字数的 2 倍词项）。
 *
 *   英文与数字按非字母数字切分并转小写。
 *
 * ⚠ 纯函数，无副作用，无平台 API —— 因此可直接在 Node 里单测。
 */
/** 一个词项及其在文本中的位置（用于高亮）。 */
export interface Token {
  term: string;
  start: number;
  end: number;
}
const CJK_START = 0x4e00;
const CJK_END = 0x9fff;
const CJK_EXT_START = 0x3400;
const CJK_EXT_END = 0x4dbf;
export function isCJK(ch: string): boolean {
  const c = ch.charCodeAt(0);
  return (c >= CJK_START && c <= CJK_END) || (c >= CJK_EXT_START && c <= CJK_EXT_END);
}
function isAlnum(ch: string): boolean {
  return /[0-9a-z]/.test(ch);
}
/**
 * 切词。
 *
 * 中文：单字 + 相邻二元组
 * 英文/数字：连续字母数字串，转小写
 *
 * @param text 原文
 * @param keepUnigram 是否保留中文单字。默认 true。
 *                    设为 false 可让索引小约一半，但"单字查询"会失效。
 */
export function tokenize(text: string, keepUnigram: boolean = true): Token[] {
  const out: Token[] = [];
  const lower = text.toLowerCase();
```

```
  const n = lower.length;
  let i = 0;
  while (i < n) {
    const ch = lower.charAt(i);
    if (isCJK(ch)) {
      // 单字
      if (keepUnigram) {
        out.push({ term: ch, start: i, end: i + 1 });
      }
      // 二元组（需要下一个字符也是 CJK）
      if (i + 1 < n && isCJK(lower.charAt(i + 1))) {
        out.push({ term: lower.substring(i, i + 2), start: i, end: i + 2 });
      }
      i += 1;
      continue;
    }
    if (isAlnum(ch)) {
      let j = i;
      while (j < n && isAlnum(lower.charAt(j))) {
        j += 1;
      }
      const term = lower.substring(i, j);
      out.push({ term: term, start: i, end: j });
      i = j;
      continue;
    }
    // 其他字符（标点、空白、emoji）不产生词项
    i += 1;
  }
  return out;
}
/** 只要词项、不要位置时用它（建索引时用）。 */
export function terms(text: string, keepUnigram: boolean = true): string[] {
  const toks = tokenize(text, keepUnigram);
  const out: string[] = [];
  for (let i = 0; i < toks.length; i++) {
    out.push(toks[i].term);
  }
  return out;
}
// ==================== lib/index.ts ====================
/**
 * 倒排索引 + BM25 打分 + 分块。
 *
 * WHY BM25 AND NOT VECTORS
 *   - 向量检索在手机上要么依赖 HMS 的 Retrieval（我们无法验证它是否存在），
 *     要么自己跑一个嵌入模型（手机上没有可用的端侧大模型）。
 *   - BM25 是**纯统计**方法：零依赖、确定性、可在 Node 里逐条断言。
 *   对一个"个人资料库"来说，BM25 的召回质量足够，而且它的打分分布
 *   **恰好就是自适应 top-k 要用的信号**（见 search.ts）。
```

```
 *
 * ⚠ 纯计算，不导入任何平台 API。
 */
import { Chunk, DocItem, Span } from './types';
import { terms, tokenize } from './tokenize';
/** BM25 参数。默认值是文献里的常用取值。 */
export const BM25_K1 = 1.5;
export const BM25_B = 0.75;
/** 分块参数。 */
export interface ChunkOptions {
  /** 每块目标字符数。中文按字符算，英文按字符算（不做语言区分，保持简单）。 */
  size: number;
  /** 相邻块的重叠字符数。避免关键词被切在边界上而漏检。 */
  overlap: number;
}
export const DEFAULT_CHUNK_OPTIONS: ChunkOptions = { size: 240, overlap: 40 };
/** 倒排表的一项：某个词项 → (块 id → 词频)。 */
interface Posting {
  tf: number;
  positions: number[];
}
export class InvertedIndex {
  /** 全部块，按加入顺序。**含已删除的墓碑**，所以不能直接用下标当"第 i 个活块"。 */
  private chunks: Chunk[] = [];
  /** 块 id → 下标。 */
  private chunkPos: Map<string, number> = new Map<string, number>();
  /** 词项 → (块下标 → Posting)。 */
  private postings: Map<string, Map<number, Posting>> = new Map<string, Map<number, Posting>>();
  /** 每个块的长度（词项数）。与 chunks 等长。 */
  private lengths: number[] = [];
  /** 墓碑标记。true 表示该块已删除，检索时跳过。 */
  private deleted: boolean[] = [];
  /** 活块的物理下标列表，顺序与加入顺序一致。`getChunk(i)` 用它映射。 */
  private liveIdx: number[] = [];
  /** 活块数。 */
  private liveCount: number = 0;
  /** 活块的长度总和，用于 avgdl。 */
  private liveLengthSum: number = 0;
  /**
   * 墓碑超过这个比例就自动压缩。
   *
   * 为什么需要：墓碑让删除变成 O(文档块数) 而不是 O(全库)，
   * 但死块会一直占着内存与遍历成本。20% 是一个折中——
   * 压缩本身是 O(全库)，太频繁就失去了增量的意义。
   */
  public static readonly COMPACT_THRESHOLD = 0.2;
  /** 已索引的**活块**数。 */
  public size(): number {
    return this.liveCount;
  }
```

```
/** 清空。重新建库时用。 */
public clear(): void {
  this.chunks = [];
  this.chunkPos = new Map<string, number>();
  this.postings = new Map<string, Map<number, Posting>>();
  this.lengths = [];
  this.deleted = [];
  this.liveIdx = [];
  this.liveCount = 0;
  this.liveLengthSum = 0;
}
/** 加入一个块。重复 id 会被忽略（幂等，便于增量建库）。 */
public addChunk(c: Chunk): boolean {
  if (this.chunkPos.has(c.id)) {
    return false;
  }
  const idx = this.chunks.length;
  this.chunks.push(c);
  this.chunkPos.set(c.id, idx);
  this.deleted.push(false);
  this.liveIdx.push(idx);
  const toks = tokenize(c.text);
  this.lengths.push(toks.length);
  this.liveCount += 1;
  this.liveLengthSum += toks.length;
  // 逐词项累积词频与位置
  for (let i = 0; i < toks.length; i++) {
    const t = toks[i];
    let perChunk = this.postings.get(t.term);
    if (perChunk === undefined) {
      perChunk = new Map<number, Posting>();
      this.postings.set(t.term, perChunk);
    }
    let p = perChunk.get(idx);
    if (p === undefined) {
      p = { tf: 0, positions: [] };
      perChunk.set(idx, p);
    }
    p.tf += 1;
    p.positions.push(t.start);
  }
  return true;
}
/**
 * 删除一份文档的全部块。
 *
 * 用**墓碑标记**而不是物理删除：物理删除会让 `chunks` 下标整体前移，
 * 所有倒排表里记录的块下标立刻全部失效，只能全量重建——
 * 那正是我们要避免的。墓碑让删除只与**该文档的块数**成正比。
 *
```

```
 * 返回被标记删除的块数。
 */
public removeDocument(docId: string): number {
  let removed = 0;
  for (let i = 0; i < this.liveIdx.length; i++) {
    const phys = this.liveIdx[i];
    if (this.chunks[phys].docId !== docId) {
      continue;
    }
    this.deleted[phys] = true;
    this.liveCount -= 1;
    this.liveLengthSum -= this.lengths[phys];
    removed += 1;
  }
  if (removed > 0) {
    this.rebuildLiveIdx();
    this.maybeCompact();
  }
  return removed;
}
/**
 * 用新文档替换旧文档的同 id 部分。
 *
 * 这是"用户重拍一张更清晰的照片"对应的操作：
 * 旧块墓碑化，新块加入。比"先删再建索引"少一次全量重建。
 *
 * 返回新加入的块数。
 */
public replaceDocument(doc: DocItem, opts: ChunkOptions = DEFAULT_CHUNK_OPTIONS): number {
  this.removeDocument(doc.id);
  // 清掉该 docId 的**全部**块 id 占位。
  //
  // ⚠ 这里曾经只遍历 liveIdx，结果漏掉了已墓碑化的旧块——
  //    它们的 id 仍留在 chunkPos 里，于是新块被当成"重复 id"而拒绝加入，
  //    表现为"替换后搜不到新内容"。
  //
  //    为什么早期测试没发现：那些用例是单文档库，删除后墓碑比例 100%
  //    会触发自动压缩，压缩顺带重建了 chunkPos，把 bug 掩盖了。
  //    多文档场景（墓碑比例低于阈值、不触发压缩）才暴露出来。
  for (let i = 0; i < this.chunks.length; i++) {
    if (this.chunks[i].docId === doc.id) {
      this.chunkPos.delete(this.chunks[i].id);
    }
  }
  return this.addDocument(doc, opts);
}
/** 活块的物理下标重排。删除后调用。 */
private rebuildLiveIdx(): void {
  const next: number[] = [];
  for (let i = 0; i < this.chunks.length; i++) {
```

```
    if (!this.deleted[i]) {
      next.push(i);
    }
  }
  this.liveIdx = next;
}
/** 墓碑比例超阈值时自动压缩。 */
private maybeCompact(): void {
  const total = this.chunks.length;
  if (total === 0) {
    return;
  }
  const deadRatio = (total - this.liveCount) / total;
  if (deadRatio >= InvertedIndex.COMPACT_THRESHOLD) {
    this.compact();
  }
}
/**
 * 物理压缩：丢掉墓碑，重建全部倒排表。
 *
 * **这是 O(全库) 的操作**，所以只在墓碑比例高时调用——
 * 频繁调用会让增量删除失去意义。
 */
public compact(): void {
  const keptChunks: Chunk[] = [];
  const keptLengths: number[] = [];
  const remap: Map<number, number> = new Map<number, number>();
  for (let i = 0; i < this.chunks.length; i++) {
    if (this.deleted[i]) {
      continue;
    }
    remap.set(i, keptChunks.length);
    keptChunks.push(this.chunks[i]);
    keptLengths.push(this.lengths[i]);
  }
  this.chunks = keptChunks;
  this.lengths = keptLengths;
  this.deleted = [];
  for (let i = 0; i < keptChunks.length; i++) {
    this.deleted.push(false);
  }
  this.liveIdx = [];
  for (let i = 0; i < keptChunks.length; i++) {
    this.liveIdx.push(i);
  }
  this.liveCount = keptChunks.length;
  this.chunkPos = new Map<string, number>();
  for (let i = 0; i < keptChunks.length; i++) {
    this.chunkPos.set(keptChunks[i].id, i);
  }
```

```
  // 重建倒排表：把旧下标重映射到新下标
  const nextPostings: Map<string, Map<number, Posting>> = new Map<string, Map<number, Posting>>();
  this.postings.forEach((perChunk: Map<number, Posting>, term: string) => {
    const remapped: Map<number, Posting> = new Map<number, Posting>();
    perChunk.forEach((p: Posting, oldIdx: number) => {
      const ni = remap.get(oldIdx);
      if (ni !== undefined) {
        remapped.set(ni, p);
      }
    });
    if (remapped.size > 0) {
      nextPostings.set(term, remapped);
    }
  });
  this.postings = nextPostings;
}
/** 墓碑数。用于界面上显示"待整理"或调试。 */
public tombstoneCount(): number {
  return this.chunks.length - this.liveCount;
}
/** 把一份资料切成块并全部加入。返回新建的块数。 */
public addDocument(doc: DocItem, opts: ChunkOptions = DEFAULT_CHUNK_OPTIONS): number {
  const chunks = chunkDocument(doc, opts);
  let added = 0;
  for (let i = 0; i < chunks.length; i++) {
    if (this.addChunk(chunks[i])) {
      added += 1;
    }
  }
  return added;
}
/** 平均块长度（词项数）。只统计活块——墓碑不该影响 avgdl。 */
public avgDocLength(): number {
  if (this.liveCount === 0) {
    return 0;
  }
  return this.liveLengthSum / this.liveCount;
}
/** 活块数。 */
public chunkCount(): number {
  return this.liveCount;
}
/**
 * 取第 i 个**活块**。
 *
 * 注意入参是"活块序号"，不是物理下标——真实下标由 `liveIdx` 映射。
 * 这样调用方（分块遍历、高亮、界面列表）不需要知道墓碑的存在。
 */
public getChunk(i: number): Chunk {
  return this.chunks[this.liveIdx[i]];
```

```
}
/**
 * BM25 打分。
 *
 * 返回**全部候选**（未截断）的得分，按降序排。
 * 不截断是刻意的：自适应 top-k 需要看完整分布才能找断层（见 search.ts）。
 */
public score(query: string): Candidate[] {
  const qTerms = terms(query);
  if (qTerms.length === 0 || this.chunks.length === 0) {
    return [];
  }
  const N = this.chunks.length;
  const avgdl = this.avgDocLength();
  // 同一查询里重复的词项只计一次
  const uniq: string[] = [];
  const seen: Map<string, boolean> = new Map<string, boolean>();
  for (let i = 0; i < qTerms.length; i++) {
    if (!seen.has(qTerms[i])) {
      seen.set(qTerms[i], true);
      uniq.push(qTerms[i]);
    }
  }
  const acc: Map<number, number> = new Map<number, number>();
  const matched: Map<number, Map<string, number>> = new Map<number, Map<string, number>>();
  for (let qi = 0; qi < uniq.length; qi++) {
    const term = uniq[qi];
    const perChunk = this.postings.get(term);
    if (perChunk === undefined) {
      continue;
    }
    const df = perChunk.size;
    // BM25 的 IDF，加 1 保证非负
    const idf = Math.log((N - df + 0.5) / (df + 0.5) + 1);
    perChunk.forEach((p: Posting, idx: number) => {
      const dl = this.lengths[idx];
      const denom = p.tf + BM25_K1 * (1 - BM25_B + (BM25_B * dl) / avgdl);
      const gain = (idf * (p.tf * (BM25_K1 + 1))) / denom;
      acc.set(idx, (acc.get(idx) ?? 0) + gain);
      let m = matched.get(idx);
      if (m === undefined) {
        m = new Map<string, number>();
        matched.set(idx, m);
      }
      m.set(term, p.tf);
    });
  }
  const out: Candidate[] = [];
  acc.forEach((score: number, idx: number) => {
    // 墓碑块必须跳过：它们已从 liveCount / avgdl 里剔除，
```

```
      // 但倒排表里仍留着记录（要等 compact 才清）。
      if (this.deleted[idx]) {
        return;
      }
      const m = matched.get(idx);
      out.push({
        index: idx,
        chunk: this.chunks[idx],
        score: score,
        matched: m !== undefined ? m : new Map<string, number>()
      });
    });
    out.sort((a: Candidate, b: Candidate) => b.score - a.score);
    return out;
  }
}
/** 一个候选（未截断）。 */
export interface Candidate {
  /** 块在索引里的下标。 */
  index: number;
  chunk: Chunk;
  score: number;
  matched: Map<string, number>;
}
/**
 * 把一份资料切成重叠的块。
 *
 * 重叠是必需的：中文关键词被切在块边界上会导致漏检，
 * 而 40 字符的重叠足以覆盖绝大多数被切断的词。
 */
export function chunkDocument(doc: DocItem, opts: ChunkOptions = DEFAULT_CHUNK_OPTIONS): Chunk[] {
  const text = doc.text;
  const out: Chunk[] = [];
  if (text.length === 0) {
    return out;
  }
  const size = Math.max(1, opts.size);
  const overlap = Math.max(0, Math.min(opts.overlap, size - 1));
  const step = Math.max(1, size - overlap);
  let seq = 0;
  let start = 0;
  while (start < text.length) {
    const end = Math.min(start + size, text.length);
    out.push({
      id: doc.id + '#' + seq.toString(),
      docId: doc.id,
      seq: seq,
      start: start,
      end: end,
      text: text.substring(start, end)
```

```
    });
    seq += 1;
    if (end >= text.length) {
      break;
    }
    start += step;
  }
  return out;
}
/** 在块内定位查询词的高亮区间。 */
export function locateSpans(chunkText: string, matched: Map<string, number>): Span[] {
  const spans: Span[] = [];
  const lower = chunkText.toLowerCase();
  matched.forEach((_tf: number, term: string) => {
    let from = 0;
    while (true) {
      const at = lower.indexOf(term, from);
      if (at < 0) {
        break;
      }
      spans.push({ start: at, end: at + term.length });
      from = at + term.length;
    }
  });
  // 按起点排序并合并重叠区间，避免界面上重复绘制
  spans.sort((a: Span, b: Span) => a.start - b.start);
  return mergeSpans(spans);
}
function mergeSpans(spans: Span[]): Span[] {
  if (spans.length <= 1) {
    return spans;
  }
  const out: Span[] = [];
  let cur = spans[0];
  for (let i = 1; i < spans.length; i++) {
    const s = spans[i];
    if (s.start <= cur.end) {
      cur = { start: cur.start, end: Math.max(cur.end, s.end) };
    } else {
      out.push(cur);
      cur = s;
    }
  }
  out.push(cur);
  return out;
}
// ==================== lib/search.ts ====================
/**
 * 检索 + 自适应 top-k 决策。
 *
```

```
 * THIS IS THE PART THAT CARRIES THE RESEARCH
 *   我们的调度器实验得出的结论是：
 *     **"剩余预算"不是决定检索深度的正确信号；"候选得分分布"才是。**
 *
 *   在资料库这个场景里，这句话变得非常具体：
 *     得分分布有明显断层 → 断层之前的是真正相关的，之后就不要再取；
 *     得分分布平坦       → 没有明显区分度，多取无益。
 *
 *   所以 `kSource: 'score-gap'` 这条路径，就是那个研究结论的产品化实现。
 *   而 `kSource: 'fixed'` 是它的对照组 —— 产品里两者都要保留，
 *   因为**用户能直观看到"固定 k"和"自适应 k"的差别**，这本身就是产品价值。
 *
 * ⚠ 纯函数，可在 Node 里逐条断言。
 */
import { InvertedIndex, Candidate, locateSpans } from './index';
import {
  Hit, Span, Chunk, ScoreDiagnostics, SearchOptions, SearchResult, BudgetInput, BudgetDecision
} from './types';
/** 相对落差的下限：小于这个值的"断层"不算断层（避免把噪声当信号）。 */
export const MIN_GAP_RATIO = 0.05;
/**
 * 计算得分分布诊断。
 *
 * `maxGap` 用**相对落差**而不是绝对差：不同查询的绝对得分量级不同，
 * 只有相对值才可比。
 */
export function diagnose(candidates: Candidate[]): ScoreDiagnostics {
  const n = candidates.length;
  if (n === 0) {
    return { candidateCount: 0, topScore: 0, mean: 0, std: 0, maxGap: 0, maxGapAt: -1 };
  }
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += candidates[i].score;
  }
  const mean = sum / n;
  let sq = 0;
  for (let i = 0; i < n; i++) {
    const d = candidates[i].score - mean;
    sq += d * d;
  }
  const std = Math.sqrt(sq / n);
  let maxGap = 0;
  let maxGapAt = -1;
  for (let i = 0; i + 1 < n; i++) {
    const hi = candidates[i].score;
    const lo = candidates[i + 1].score;
    if (hi <= 0) {
      continue;
    }
```

```
    const ratio = (hi - lo) / hi;
    if (ratio > maxGap) {
      maxGap = ratio;
      maxGapAt = i;
    }
  }
  return {
    candidateCount: n,
    topScore: candidates[0].score,
    mean: mean,
    std: std,
    maxGap: maxGap,
    maxGapAt: maxGapAt
  };
}
/**
 * 由得分分布决定 k。
 *
 * 规则：
 *   1. 找到相邻得分的最大相对落差位置 g
 *   2. 若该落差 >= MIN_GAP_RATIO，取 k = g + 1（断层之前的全部保留）
 *   3. 否则说明分布平坦、没有明显相关边界 → 取 maxK（反正区分不出来）
 *   4. 最后夹到 [minK, maxK] 与候选总数内
 */
export function decideKByScoreGap(
  candidates: Candidate[],
  minK: number,
  maxK: number,
  diag?: ScoreDiagnostics
): number {
  const n = candidates.length;
  if (n === 0) {
    return 0;
  }
  const d = diag !== undefined ? diag : diagnose(candidates);
  let k: number;
  if (d.maxGapAt >= 0 && d.maxGap >= MIN_GAP_RATIO) {
    k = d.maxGapAt + 1;
  } else {
    // 分布平坦：没有可信的断层，用上界
    k = maxK;
  }
  if (k < minK) {
    k = minK;
  }
  if (k > maxK) {
    k = maxK;
  }
  if (k > n) {
    k = n;
```

```
  }
  return k;
}
/**
 * 把同一文档中**重叠或首尾相接**的命中块合并成一条。
 *
 * WHY THIS EXISTS
 *   分块有 40 字符重叠，所以一段连续文字里的关键词很容易同时命中相邻的几块。
 *   对用户来说那就是"同一段话"，显示成 3 条是纯噪音。
 *
 * HOW IT MERGES
 *   · 文本：直接按偏移拼回去。chunkDocument 的 text 是 text.substring(start, end)，
 *     所以当后块起点落在前块范围内时，只需补上超出部分。
 *   · 得分：取**最高分**，不累加。累加会让"被切成很多块的长文档"凭空得高分，
 *     那是把分块策略的副作用当成相关度。
 *   · 高亮：后块的 span 需要减去前块起点，换算到合并后的坐标系。
 *   · 命中词频：相加，因为合并区域里确实出现了那么多次。
 *
 *   **不合并不同文档**：跨文档拼在一起没有意义，用户也不会那样读。
 *
 * ⚠ 只在 ranges 真正相接或重叠时合并。中间有空隙就不合并 ——
 *    否则会把"第 1 段和第 20 段"合成一条，看起来像连续引用，实际不是。
 */
export function mergeAdjacentHits(hits: Hit[]): Hit[] {
  if (hits.length <= 1) {
    return hits;
  }
  // 记录每条属于哪个合并组，以及组内成员的原始名次（用于保持排序）
  const groups: Hit[] = [];
  const groupRank: number[] = [];
  for (let i = 0; i < hits.length; i++) {
    const h = hits[i];
    let target = -1;
    // 找同一文档且范围相接/重叠的已有组
    for (let g = 0; g < groups.length; g++) {
      const cur = groups[g];
      if (cur.chunk.docId !== h.chunk.docId) {
        continue;
      }
      if (h.chunk.start <= cur.chunk.end && cur.chunk.start <= h.chunk.end) {
        target = g;
        break;
      }
    }
    if (target < 0) {
      const copy: Hit = {
        chunk: h.chunk,
        score: h.score,
        matched: new Map<string, number>(h.matched),
        spans: h.spans.slice()
```

```
  };
  if (h.mergedCount !== undefined) {
    copy.mergedCount = h.mergedCount;
  }
  groups.push(copy);
  groupRank.push(i);
  continue;
}
const g0 = groups[target];
const base = g0.chunk;
const add = h.chunk;
// 拼文本：add 覆盖 [add.start, add.end)，base 已覆盖 [base.start, base.end)
const baseLen = base.text.length;
const baseEnd = base.start + baseLen;
const tailFrom = baseEnd - add.start;
let newText = base.text;
if (tailFrom >= 0 && tailFrom < add.text.length) {
  newText = base.text + add.text.substring(tailFrom);
} else if (tailFrom < 0) {
  // 中间有空隙：不合并，另起一组（正常不会走到，这里防御）
  groups.push({
    chunk: add, score: h.score,
    matched: new Map<string, number>(h.matched), spans: h.spans.slice()
  });
  groupRank.push(i);
  continue;
}
// 高亮坐标换算
const shift = add.start - base.start;
const spans = g0.spans.slice();
for (let s = 0; s < h.spans.length; s++) {
  const nv: Span = { start: h.spans[s].start + shift, end: h.spans[s].end + shift };
  spans.push(nv);
}
spans.sort((a: Span, b: Span) => a.start - b.start);
// 词频相加
const matched = g0.matched;
h.matched.forEach((tf: number, term: string) => {
  matched.set(term, (matched.get(term) ?? 0) + tf);
});
const mergedChunk: Chunk = {
  id: base.docId + '#' + base.seq.toString() + '-' + add.seq.toString(),
  docId: base.docId,
  seq: base.seq,
  start: base.start,
  end: Math.max(baseEnd, add.end),
  text: newText
};
const merged: Hit = {
  chunk: mergedChunk,
```

```
      score: Math.max(g0.score, h.score),
      matched: matched,
      spans: spans,
      mergedCount: (g0.mergedCount ?? 1) + (h.mergedCount ?? 1)
    };
    groups[target] = merged;
  }
  // 按各组里最好的原始名次排序，保持与合并前一致的顺序
  const order: number[] = [];
  for (let i = 0; i < groups.length; i++) {
    order.push(i);
  }
  order.sort((a: number, b: number) => groupRank[a] - groupRank[b]);
  const out: Hit[] = [];
  for (let i = 0; i < order.length; i++) {
    out.push(groups[order[i]]);
  }
  return out;
}
/**
 * 检索主入口。
 *
 * `options.topK > 0` → 固定 k（对照组）
 * `options.topK <= 0` → 由得分分布自适应（我们的方法）
 * `options.mergeAdjacent` → 选出 top-k 之后，把同文档相接的块合并
 */
export function search(index: InvertedIndex, query: string, options: SearchOptions): SearchResult {
  const candidates = index.score(query);
  const diag = diagnose(candidates);
  let k: number;
  let kSource: 'fixed' | 'score-gap';
  if (options.topK > 0) {
    k = options.topK;
    kSource = 'fixed';
  } else {
    k = decideKByScoreGap(candidates, options.minK, options.maxK, diag);
    kSource = 'score-gap';
  }
  if (k > candidates.length) {
    k = candidates.length;
  }
  const hits: Hit[] = [];
  for (let i = 0; i < k; i++) {
    const c = candidates[i];
    hits.push({
      chunk: c.chunk,
      score: c.score,
      matched: c.matched,
      spans: locateSpans(c.chunk.text, c.matched)
    });
```

```
  }
  if (options.mergeAdjacent === true) {
    return { hits: mergeAdjacentHits(hits), k: k, kSource: kSource, diagnostics: diag };
  }
  return { hits: hits, k: k, kSource: kSource, diagnostics: diag };
}
/**
 * 由资源预算决定"还能不能多取几条"。
 *
 * 注意这与上面的 k 决策是**两件事**：
 *   - `decideKByScoreGap` 回答"该取几条"（信息层面的问题，看得分分布）
 *   - `decideBudget`      回答"还取得起吗"（资源层面的问题，看延迟/内存/电量）
 *
 * 这正是我们研究主张的另一半：**两个问题需要两个不同的信号**。
 * 把它们混成一个"剩余预算"信号，就会得到实验里那个被严格支配的结果。
 */
export function decideBudget(input: BudgetInput, avgChunkChars: number): BudgetDecision {
  // 每条命中大约要一次排序 + 一次高亮定位，按经验折算成时间
  const msPerHit = 0.6;
  const kbPerHit = (avgChunkChars * 2) / 1024 + 0.5;
  const byLatency = Math.floor(input.latencyBudgetMs / msPerHit);
  const byMemory = Math.floor(input.memoryBudgetKb / kbPerHit);
  let k = Math.min(byLatency, byMemory);
  const reasons: string[] = [];
  if (byLatency <= byMemory) {
    reasons.push('受时间预算限制');
  } else {
    reasons.push('受内存预算限制');
  }
  // 低电量：主动降级，避免后台建库/检索耗电
  if (input.batteryPercent <= 15) {
    k = Math.floor(k / 2);
    reasons.push('电量偏低，已降级');
  }
  if (k < 1) {
    k = 1;
    reasons.push('已压到最小 1 条');
  }
  return { k: k, reason: reasons.join('；') };
}
// ==================== lib/pipeline.ts ====================
/**
 * 流水线编排 —— 导入 → 分块 → 建索引 → 检索。
 *
 * THIS IS THE PART THE PPT CALLS "本项目的技术含量所在"
 *   开题 PPT 的 P9 写着：
 *     "① 本地索引与检索（切词 / 倒排 / BM25 / 高亮定位）——纯算法，可脱离设备做单元测试
 *      ② 端侧能力编排与调度——本项目的技术含量所在"
 *
 *   ① 在 index.ts / search.ts 里。② 就在这个文件。
```

```
 *
 *   编排的核心不是"把函数串起来"，而是**每一步都要做一次带信号的决策**：
 *
 *     感知环节  → 识别置信度 → 收下 / 提示重拍 / 丢弃        （信号：图像质量）
 *     索引环节  → 文档长度   → 分块大小 / 是否延迟建索引     （信号：内容规模）
 *     检索环节  → 得分分布   → 取几条                        （信号：得分断层）
 *     资源环节  → 延迟/内存/电量 → 还能取几条                （信号：资源）
 *
 *   **这四条用的是四个不同的信号。** 若把它们压成一个"剩余预算"，
 *   就退化成了我们实验里那个被每个固定策略严格支配的方案。
 *
 * ⚠ 纯 TS：不导入任何平台 Kit。OCR 由外部注入（见 ocr.ts）。
 */
import { InvertedIndex, chunkDocument, ChunkOptions, DEFAULT_CHUNK_OPTIONS } from './index';
import { search, decideBudget } from './search';
import { DocStore } from './store';
import { OcrEngine, judgeOcr, OcrVerdict, OcrQualityPolicy, DEFAULT_OCR_POLICY } from './ocr';
import {
  BudgetInput, Hit, SearchOptions, DocItem, DocSource, ScoreDiagnostics
} from './types';
/** 导入一份资料的输入。 */
export interface ImportInput {
  /** 用户可见的标题。 */
  title: string;
  /** 来源。决定是否需要 OCR。 */
  source: DocSource;
  /** 原始文件位置。 */
  uri: string;
  /**
   * 已经有文本时直接给（纯文本资料、或用户手工粘贴的笔记）。
   * 为空则走 OCR。
   */
  text?: string;
}
/** 导入的结果。 */
export interface ImportOutcome {
  /** 是否成功入库。 */
  ok: boolean;
  /** OCR 判定（纯文本导入时恒为 'accept'）。 */
  verdict: OcrVerdict;
  /** 入库后的文档 id（失败时为空串）。 */
  docId: string;
  /** 新建的块数。 */
  chunks: number;
  /** 给用户看的一句话（成功或失败原因）。 */
  message: string;
}
/** 一次查询的结果，含全部决策依据。 */
export interface QueryOutcome {
  hits: Hit[];
```

```
  k: number;
  kSource: 'fixed' | 'score-gap';
  diagnostics: ScoreDiagnostics;
  /** 资源预算给出的上限。 */
  budgetK: number;
  /** 资源预算的理由（"受时间预算限制"等）。 */
  budgetReason: string;
  /** 最终 k 是受信息信号还是资源信号限制。 */
  limitedBy: 'information' | 'resource';
}
/**
 * 资料库服务 —— 应用层唯一需要交互的入口。
 *
 * 把 store / index / ocr 三者组合起来，并保证：
 *   - 文档变更后索引与文档库保持一致
 *   - 每次检索都返回**决策依据**（便于界面向用户解释）
 */
export class LibraryService {
  private store: DocStore;
  private index: InvertedIndex;
  private ocr: OcrEngine;
  private chunkOpts: ChunkOptions;
  private ocrPolicy: OcrQualityPolicy;
  private seq: number;
  constructor(
    ocr: OcrEngine,
    chunkOpts: ChunkOptions = DEFAULT_CHUNK_OPTIONS,
    ocrPolicy: OcrQualityPolicy = DEFAULT_OCR_POLICY
  ) {
    this.store = new DocStore();
    this.index = new InvertedIndex();
    this.ocr = ocr;
    this.chunkOpts = chunkOpts;
    this.ocrPolicy = ocrPolicy;
    this.seq = 0;
  }
  /** 文档数。 */
  public docCount(): number {
    return this.store.size();
  }
  /** 索引块数。 */
  public chunkCount(): number {
    return this.index.chunkCount();
  }
  /** OCR 能力是否可用。**界面在提供"拍照导入"入口前必须先问这个。** */
  public ocrAvailable(): boolean {
    return this.ocr.available();
  }
  public ocrName(): string {
    return this.ocr.name();
```

```
}
/**
 * 导入一份资料。
 *
 * 决策点：OCR 置信度决定"收下 / 提示重拍 / 丢弃"。
 * 这是"感知环节由图像质量驱动"的具体实现。
 */
public async importDocument(input: ImportInput): Promise<ImportOutcome> {
  let text = input.text !== undefined ? input.text : '';
  let verdict: OcrVerdict = 'accept';
  if (text.length === 0) {
    // 没有现成文本 → 走 OCR
    if (!this.ocr.available()) {
      return {
        ok: false,
        verdict: 'reject',
        docId: '',
        chunks: 0,
        message: '当前设备没有可用的 OCR 能力，请改用「粘贴文本」导入'
      };
    }
    const res = await this.ocr.recognize(input.uri);
    verdict = judgeOcr(res, this.ocrPolicy);
    if (verdict === 'reject') {
      return {
        ok: false,
        verdict: verdict,
        docId: '',
        chunks: 0,
        message: res.error !== undefined ? res.error : '识别质量过低，未入库'
      };
    }
    text = res.text;
  }
  if (text.length === 0) {
    return { ok: false, verdict: 'reject', docId: '', chunks: 0, message: '内容为空' };
  }
  this.seq += 1;
  const docId = 'd' + this.seq.toString();
  const doc: DocItem = {
    id: docId,
    title: input.title,
    source: input.source,
    uri: input.uri,
    addedAt: 0,
    text: text
  };
  this.store.put(doc);
  // 决策点：内容规模决定分块大小
  const opts = this.chunkOptionsFor(text.length);
```

```
  const added = this.index.addDocument(doc, opts);
  const base = verdict === 'retake'
    ? '已入库，但识别置信度偏低，建议重拍以获得更好的检索效果'
    : '已入库';
  return {
    ok: true,
    verdict: verdict,
    docId: docId,
    chunks: added,
    message: base + '（' + added.toString() + ' 个片段）'
  };
}
/**
 * 由文档长度决定分块参数。
 *
 * 为什么不是固定值：短文档（一页笔记）按 240 字符切会切成一个块，
 * 而长文档（几十页 PDF）按 240 切会产生上千个块、索引膨胀。
 * **内容规模是一个独立信号**——它只影响分块，不该影响检索深度。
 */
private chunkOptionsFor(textLen: number): ChunkOptions {
  if (textLen <= 1000) {
    return { size: Math.max(120, textLen), overlap: 0 };
  }
  if (textLen <= 20000) {
    return { size: 240, overlap: 40 };
  }
  // 长文档：块更大以减少块数，保留重叠避免切断
  return { size: 400, overlap: 60 };
}
/**
 * 查询。
 *
 * **两个信号各管一件事**：
 *   - 得分分布 → 该取几条（信息层面）
 *   - 延迟/内存/电量 → 还取得起吗（资源层面）
 * 最终 k 取两者的较小值，并报告是谁在限制。
 */
public query(q: string, budget: BudgetInput, options: SearchOptions): QueryOutcome {
  const r = search(this.index, q, options);
  const bd = decideBudget(budget, this.averageChunkChars());
  let k = r.k;
  let limitedBy: 'information' | 'resource' = 'information';
  if (bd.k < r.k) {
    k = bd.k;
    limitedBy = 'resource';
  }
  const hits: Hit[] = r.hits.slice(0, k);
  return {
    hits: hits,
    k: k,
```

```
    kSource: r.kSource,
    diagnostics: r.diagnostics,
    budgetK: bd.k,
    budgetReason: bd.reason,
    limitedBy: limitedBy
  };
}
/**
 * 实际的块平均字符数。
 *
 * 早先这里写的是拿一个固定参数去推，那是错的——块大小随文档长度变，
 * 必须从**真实块**上量。这条注释留着，是因为"看起来合理的估计"
 * 正是这个项目里反复出错的形态。
 */
public averageChunkChars(): number {
  const n = this.index.chunkCount();
  if (n === 0) {
    return 0;
  }
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += this.index.getChunk(i).text.length;
  }
  return sum / n;
}
/** 序列化整个资料库（只存文档，索引在 load 时重建）。 */
public save(): string {
  return this.store.save();
}
/**
 * 恢复资料库并重建索引。
 *
 * 返回被跳过的坏条目数。**rebuildIndex 是必需的**——
 * 存档里只有文档，没有索引（见 store.ts 的设计决定）。
 */
public load(json: string): number {
  const skipped = this.store.load(json);
  this.seq = this.store.size();
  this.rebuildIndex();
  return skipped;
}
/** 从文档库重建索引。返回重建的块数。 */
public rebuildIndex(): number {
  this.index.clear();
  const docs = this.store.all();
  let n = 0;
  for (let i = 0; i < docs.length; i++) {
    const d = docs[i];
    n += this.index.addDocument(d, this.chunkOptionsFor(d.text.length));
  }
```

```
  return n;
}
/**
 * 删除一份资料。
 *
 * **不再全量重建索引**：早先这里写的是 `store.remove` + `rebuildIndex()`，
 * 删除一份文档要把全库重新切词、重新建倒排表。
 * 1000 份文档时删 10 份就是 10 次全量重建。
 *
 * 现在走 `InvertedIndex.removeDocument()` 的墓碑路径，
 * 代价只与该文档的块数成正比；墓碑累积到阈值时引擎会自动压缩。
 */
public removeDocument(id: string): boolean {
  const ok = this.store.remove(id);
  if (ok) {
    this.index.removeDocument(id);
  }
  return ok;
}
/**
 * 替换一份资料（用户重拍了照片、或重新粘贴了文本）。
 *
 * 比"先 removeDocument 再 importDocument"少一次全量重建，
 * 而且能保持文档在列表中的位置。
 */
public async replaceDocument(id: string, input: ImportInput): Promise<ImportOutcome> {
  const old = this.store.get(id);
  if (old === undefined) {
    return { ok: false, verdict: 'reject', docId: '', chunks: 0, message: '要替换的文档不存在' };
  }
  let text = input.text !== undefined ? input.text : '';
  let verdict: OcrVerdict = 'accept';
  if (text.length === 0) {
    if (!this.ocr.available()) {
      return {
        ok: false, verdict: 'reject', docId: '', chunks: 0,
        message: '当前设备没有可用的 OCR 能力，请改用「粘贴文本」导入'
      };
    }
    const res = await this.ocr.recognize(input.uri);
    verdict = judgeOcr(res, this.ocrPolicy);
    if (verdict === 'reject') {
      return {
        ok: false, verdict: verdict, docId: '', chunks: 0,
        message: res.error !== undefined ? res.error : '识别质量过低，未替换'
      };
    }
    text = res.text;
  }
  if (text.length === 0) {
```

```
      return { ok: false, verdict: 'reject', docId: '', chunks: 0, message: '内容为空' };
    }
    const doc: DocItem = {
      id: id,
      title: input.title,
      source: input.source,
      uri: input.uri,
      addedAt: old.addedAt,
      text: text
    };
    this.store.put(doc);
    const added = this.index.replaceDocument(doc, this.chunkOptionsFor(text.length));
    return {
      ok: true, verdict: verdict, docId: id, chunks: added,
      message: verdict === 'retake' ? '已替换，但识别置信度偏低，建议重拍' : '已替换'
    };
  }
  /** 墓碑数（死块）。界面可据此提示"索引需要整理"。 */
  public tombstoneCount(): number {
    return this.index.tombstoneCount();
  }
  /** 手动压缩索引。一般不需要——引擎会在墓碑超阈值时自动压缩。 */
  public compactIndex(): void {
    this.index.compact();
  }
  /** 训练/演示用：列出全部文档。 */
  public documents(): DocItem[] {
    return this.store.all();
  }
}
// ==================== lib/ocr.ts ====================
/**
 * OCR 抽象层。
 *
 * WHY AN INTERFACE INSTEAD OF CALLING THE KIT DIRECTLY
 *   这是整个工程最重要的一条架构决定。
 *
 *   我们实测过一个真实的崩溃：应用静态 `import` 了 HMS 的 A2A Kit，
 *   而设备上没有对应的 HSP 时，ArkTS 运行时会在**模块加载期**直接
 *   `abort` 整个进程 —— 不是"功能不可用"，是"应用起不来"。
 *   （崩溃日志：`load hsp failed, hsp name: com.huawei.hmsapp.hiai.hsp/agentKitHsp`）
 *
 *   所以 OCR 在这里是一个**接口**，而不是一个 import。
 *   真正的实现（调用 Vision Kit）由上层注入，并且**必须在使用前先探测能力是否存在**。
 *   这样：
 *     1. 本模块可以在 Node 里被单测，不需要设备
 *     2. 设备上没有 OCR 时，应用仍然能启动（只是这个能力降级）
 *
 * ⚠ 本文件不导入任何 HarmonyOS Kit。这是刻意的。
 */
```

```
/** 一次 OCR 的结果。 */
export interface OcrResult {
  /** 识别出的纯文本。失败时为空串。 */
  text: string;
  /** 平均置信度 0-1。0 表示完全不可信或识别失败。 */
  confidence: number;
  /** 用时（毫秒），用于预算决策。 */
  elapsedMs: number;
  /** 失败原因；成功时为 undefined。 */
  error?: string;
  /**
   * 字符覆盖率：识别出的字数 ÷ 由行几何估算的应有字数。
   *
   * WHY THIS FIELD EXISTS
   *   我们用 12 种退化条件实测过（见 deliverables/演示素材/L7 OCR 质量测试/），
   *   OCR 的失败方式是**漏认**而不是**错认**：
   *   在「小字+模糊」那张图上，召回只有 28%，但精确率是 100% ——
   *   认出来的每个字都对，只是认得太少。
   *
   *   所以真正有效的失败信号是「认得够不够多」，「认得多准」反而不是瓶颈。
   *   这个字段就是那个信号：明显小于 1 说明有大片文字没被认出来。
   *
   *   估算方式：中日韩字形近似正方形，所以一行"N 个字"的行宽约为 N 倍行高，
   *   于是 应有字数 ≈ 行宽 ÷ 行高。
   *   拉丁字符比汉字窄，会让这个估算偏小、coverage 偏大 ——
   *   即**倾向低估问题**。这是刻意选的保守方向：
   *   宁可漏报，也不要因为英文行多算而误判为"质量差"。
   */
  coverage?: number;
  /** 参与覆盖率估算的行数；为 0 表示引擎没给出几何信息。 */
  lineCount?: number;
  /**
   * 图片锐度：相邻像素灰度差的平均绝对值（0-255 量级）。
   *
   * WHY THIS FIELD EXISTS — 一次失败的尝试
   *   我最初想用**覆盖率**做置信度：用 OCR 检测到的行框算"这一行该有多少字"，
   *   再和实际认出的字数比。**实测证明完全无效** ——
   *   最差的那张图（召回 28%）覆盖率和正常图几乎一样（1.74 vs 1.72）。
   *
   *   失效原因很根本：expected 是从**检测到的行框**算出来的。
   *   如果 OCR 整行都没检测到，那一行的框也不存在 ——
   *   实际字数和应有字数**一起缩小**，比值自我归一化，对漏认天然免疫。
   *   **检测器的盲区，在它自己的输出里是看不见的。**
   *
   *   所以要判断"这张图是不是糊到不能用"，必须用一个
   *   **完全不依赖 OCR 输出的**参照。锐度直接读像素，就是这个参照。
   *
   * ⚠ 它是相对指标，不是标定的物理量：不同分辨率下取值会变，
   *    阈值必须按设备实测确定，不能当通用常数。
   */
```

```
  sharpness?: number;
}
/** OCR 引擎。真实实现由 ArkTS 侧提供。 */
export interface OcrEngine {
  /** 引擎名，用于界面显示与排错。 */
  name(): string;
  /** 这个引擎在当前设备上是否可用。**调用方必须先问这个，再决定用不用。** */
  available(): boolean;
  /** 识别一张图片。 */
  recognize(imageUri: string): Promise<OcrResult>;
}
/**
 * 假 OCR 引擎 —— 用预设文本代替真实识别。
 *
 * 用途有三个，都不是"应付测试"：
 *   1. **让检索流水线在没有设备的机器上可端到端跑通**（老师要的"可核实"）
 *   2. 在设备 OCR 不可用时作为**降级占位**，让其余功能仍可演示
 *   3. 造可控的"识别质量差"场景，用来测调度模块的降级逻辑
 */
export class FakeOcrEngine implements OcrEngine {
  private table: Map<string, OcrResult>;
  private defaultText: string;
  private failUris: Set<string>;
  /**
   * @param defaultText 没有预设时返回的文本
   */
  constructor(defaultText: string = '') {
    this.table = new Map<string, OcrResult>();
    this.defaultText = defaultText;
    this.failUris = new Set<string>();
  }
  public name(): string {
    return 'fake-ocr';
  }
  public available(): boolean {
    return true;
  }
  /** 预设某个 URI 的识别结果。 */
  public preset(uri: string, text: string, confidence: number): FakeOcrEngine {
    this.table.set(uri, { text: text, confidence: confidence, elapsedMs: 10 });
    return this;
  }
  /** 预设某个 URI 识别失败。 */
  public presetFailure(uri: string): FakeOcrEngine {
    this.failUris.add(uri);
    return this;
  }
  public recognize(imageUri: string): Promise<OcrResult> {
    if (this.failUris.has(imageUri)) {
      return Promise.resolve({
```

```
        text: '',
        confidence: 0,
        elapsedMs: 5,
        error: 'fake: 预设的识别失败'
      });
    }
    const hit = this.table.get(imageUri);
    if (hit !== undefined) {
      return Promise.resolve(hit);
    }
    return Promise.resolve({
      text: this.defaultText,
      confidence: this.defaultText.length > 0 ? 0.5 : 0,
      elapsedMs: 5,
      error: this.defaultText.length > 0 ? undefined : 'fake: 无预设文本'
    });
  }
}
/**
 * 一个永远不可用的引擎 —— 用于模拟"设备没有 OCR 能力"。
 *
 * 它的存在是为了让"能力缺失时应用仍能启动"这条设计**可被测试**，
 * 而不是只在文档里承诺。
 */
export class UnavailableOcrEngine implements OcrEngine {
  private why: string;
  constructor(why: string = '设备不具备 OCR 能力') {
    this.why = why;
  }
  public name(): string {
    return 'unavailable-ocr';
  }
  public available(): boolean {
    return false;
  }
  public recognize(_imageUri: string): Promise<OcrResult> {
    return Promise.resolve({
      text: '',
      confidence: 0,
      elapsedMs: 0,
      error: this.why
    });
  }
}
/**
 * 根据识别置信度决定"要不要提示用户重拍"。
 *
 * 这是三个信号里的**感知环节信号**：图像质量 → 是否重拍。
 * 对应研究主张里"感知（OCR）应当由图像质量驱动"这一条。
 */
```

```
export interface OcrQualityPolicy {
  /** 低于这个置信度就提示重拍。 */
  retakeThreshold: number;
  /** 低于这个置信度就认为识别失败，不应入库。 */
  rejectThreshold: number;
}
export const DEFAULT_OCR_POLICY: OcrQualityPolicy = {
  retakeThreshold: 0.6,
  rejectThreshold: 0.2
};
export type OcrVerdict = 'accept' | 'retake' | 'reject';
/** 掉一个 OCR 结果该不该收、该不该提示重拍。 */
export function judgeOcr(result: OcrResult, policy: OcrQualityPolicy = DEFAULT_OCR_POLICY): OcrVerdict {
  if (result.error !== undefined || result.text.length === 0) {
    return 'reject';
  }
  if (result.confidence < policy.rejectThreshold) {
    return 'reject';
  }
  if (result.confidence < policy.retakeThreshold) {
    return 'retake';
  }
  return 'accept';
}
// ==================== selfcheck/Capability.ets ====================
/**
 * 设备能力探测 —— 用 `canIUse` 查系统能力是否存在。
 *
 * WHY canIUse AND NOT A DIRECT IMPORT
 *   这个工程在早期版本里踩过一个真实的坑：
 *   应用静态 `import { createA2AServer } from '@hms.ai.A2A'`，
 *   而设备上没有对应的 HSP 时，ArkTS 运行时会在**模块加载期**直接 abort 整个进程——
 *   不是"功能不可用"，是"应用起不来"（崩溃日志里是 LoadJSPandaFile ... abort）。
 *
 *   所以探测能力必须用 `canIUse`：它是全局函数，**不需要 import 任何模块**，
 *   因此不存在把应用拖死的风险。
 *
 *   `canIUse` 返回 true 只说明"这个 syscap 被声明可用"，
 *   不代表 API 一定调得通；但返回 false 就**一定别去 import**。
 */
/** 一条能力探测结果。 */
export interface CapabilityProbe {
  /** 界面上显示的名字。 */
  label: string;
  /** syscap 全名。 */
  syscap: string;
  /** canIUse 的结果。 */
  available: boolean;
  /** 这个能力在本项目里用来做什么。 */
  purpose: string;
```

```
  }
  if (typeof d.addedAt !== 'number' || !isFinite(d.addedAt)) {
    return false;
  }
  if (!isValidSource(d.source)) {
    return false;
  }
  return true;
}
export function isValidSource(s: DocSource): boolean {
  return s === 'camera' || s === 'album' || s === 'file' || s === 'note';
}
/** 生成一个文档 id。时间戳 + 序号，够用且可读。 */
export function makeDocId(seq: number, now: number): string {
  return 'd' + now.toString(36) + '-' + seq.toString(36);
}
// ==================== lib/librarystorage.ets ====================
/**
 * 资料库的本地持久化。
 *
 * WHY THIS FILE EXISTS — 一个我自己造成的问题
 *   核心库里早就有 `DocStore.save()/load()`，自测里也在用，
 *   但**页面从来没调用过它们**。后果是：
 *
 *     应用一关，资料全没了；下次打开因为库是空的，
 *     `seedDemo()` 又会塞回那份示例 —— 看起来像"正常"，其实是每次都在重新开始。
 *
 *   一个"资料库"关掉就忘事，不是小瑕疵，是功能不成立。
 *   所以这一步补上：存档、启动时读回、任何改动后立即落盘。
 *
 * WHY SAVE ON EVERY CHANGE INSTEAD OF ON EXIT
 *   HarmonyOS 的 UIAbility 在后台可能被系统直接回收，`onDestroy` 不保证被调用。
 *   "退出时保存"在这种模型下是不可靠的。
 *   改动本来就很少（导入/删除），每次落盘的开销可以忽略。
 *
 * WHY NOT STORE THE INDEX
 *   只存原始文档，索引在启动时重建（见 core 的 README）。
 *   索引格式一改旧档就废；重建在常见规模下是毫秒级。
 */
import { fileIo } from '@kit.CoreFileKit';
import { common } from '@kit.AbilityKit';
/** 存档文件名。放在应用沙箱的 files 目录，卸载即清除。 */
const FILE_NAME = 'library.json';
/** 存档文件的绝对路径。 */
export function libraryPath(ctx: common.UIAbilityContext): string {
  return ctx.filesDir + '/' + FILE_NAME;
}
/** 把一段文本写进沙箱。返回是否成功。 */
export function writeText(ctx: common.UIAbilityContext, name: string, text: string): boolean {
  let f: fileIo.File | undefined = undefined;
```

```
  try {
    const p = ctx.filesDir + '/' + name;
    // 先删再建：避免旧内容比新内容长时尾部残留
    try {
      fileIo.unlinkSync(p);
    } catch (e) {
      // 文件不存在是正常的，忽略
    }
    f = fileIo.openSync(p, fileIo.OpenMode.CREATE | fileIo.OpenMode.READ_WRITE);
    fileIo.writeSync(f.fd, text);
    return true;
  } catch (e) {
    return false;
  } finally {
    if (f !== undefined) {
      try {
        fileIo.closeSync(f);
      } catch (e) {
        // 关不掉不影响写入结果
      }
    }
  }
}
/**
 * 从沙箱读一段文本。
 *
 * 返回空串表示"还没有存档"（首次启动），调用方据此决定要不要放示例资料。
 * **不要把"读不到"和"读到了但是空的"混为一谈** —— 前者该放示例，后者不该。
 */
export function readText(ctx: common.UIAbilityContext, name: string): string {
  let f: fileIo.File | undefined = undefined;
  try {
    const p = ctx.filesDir + '/' + name;
    f = fileIo.openSync(p, fileIo.OpenMode.READ_ONLY);
    const stat: fileIo.Stat = fileIo.statSync(f.fd);
    if (stat.size <= 0) {
      return '';
    }
    const buf: ArrayBuffer = new ArrayBuffer(stat.size);
    const n = fileIo.readSync(f.fd, buf);
    if (n <= 0) {
      return '';
    }
    return decodeUtf8(new Uint8Array(buf, 0, n));
  } catch (e) {
    return '';
  } finally {
    if (f !== undefined) {
      try {
        fileIo.closeSync(f);
```

```
      } catch (e) {
        // 忽略
      }
    }
  }
}
/** 存档文件是否存在。 */
export function hasArchive(ctx: common.UIAbilityContext): boolean {
  try {
    return fileIo.accessSync(libraryPath(ctx));
  } catch (e) {
    return false;
  }
}
/**
 * 按 URI 读一个文本文件（用于系统文件选择器返回的 txt / md）。
 *
 * 与 readText 的区别：readText 只读我们自己沙箱里的固定文件，
 * 这里要读用户通过选择器挑的任意位置的文件。
 * 两者共用同一套 UTF-8 解码逻辑。
 */
export function readTextByUri(uri: string): string {
  let f: fileIo.File | undefined = undefined;
  try {
    f = fileIo.openSync(uri, fileIo.OpenMode.READ_ONLY);
    const stat: fileIo.Stat = fileIo.statSync(f.fd);
    // 单文件上限 4MB：超过的不导入，避免一次读进内存把应用撑住
    const MAX = 4 * 1024 * 1024;
    if (stat.size <= 0 || stat.size > MAX) {
      return '';
    }
    const buf: ArrayBuffer = new ArrayBuffer(stat.size);
    const n = fileIo.readSync(f.fd, buf);
    if (n <= 0) {
      return '';
    }
    return decodeUtf8(new Uint8Array(buf, 0, n));
  } catch (e) {
    return '';
  } finally {
    if (f !== undefined) {
      try {
        fileIo.closeSync(f);
      } catch (e) {
        // 忽略
      }
    }
  }
}
/**
```

```
 * UTF-8 解码。
 *
 * 为什么自己写而不依赖 TextDecoder：
 *   这个解码器只处理"合法 UTF-8"，遇到坏字节就跳过，
 *   不会抛异常打断整个导入流程。对用户从别处拷来的文本文件更稳。
 */
function decodeUtf8(view: Uint8Array): string {
  let out = '';
  let i = 0;
  while (i < view.length) {
    const b0 = view[i];
    if (b0 < 0x80) {
      out += String.fromCharCode(b0);
      i += 1;
    } else if (b0 >= 0xF0 && i + 3 < view.length) {
      const cp = ((b0 & 0x07) << 18) | ((view[i + 1] & 0x3F) << 12)
        | ((view[i + 2] & 0x3F) << 6) | (view[i + 3] & 0x3F);
      out += String.fromCodePoint(cp);
      i += 4;
    } else if (b0 >= 0xE0 && i + 2 < view.length) {
      const cp = ((b0 & 0x0F) << 12) | ((view[i + 1] & 0x3F) << 6) | (view[i + 2] & 0x3F);
      out += String.fromCharCode(cp);
      i += 3;
    } else if (b0 >= 0xC0 && i + 1 < view.length) {
      const cp = ((b0 & 0x1F) << 6) | (view[i + 1] & 0x3F);
      out += String.fromCharCode(cp);
      i += 2;
    } else {
      i += 1;
    }
  }
  return out;
}
// ==================== lib/realocr.ets ====================
/**
 * 真实 OCR 引擎（CoreVisionKit）。
 *
 * ⚠ 这个文件**静态导入了 HMS 的 CoreVisionKit**。
 *
 *   为什么这是安全的：我们先用 `canIUse('SystemCapability.AI.OCR.TextRecognition')`
 *   在设备上实测过，返回 **true**（截图见 deliverables/演示素材/L3 真机验证/）。
 *
 *   为什么仍然要谨慎：早期版本静态导入 `@hms.ai.A2A` 时，因为设备缺少对应 HSP，
 *   应用在**模块加载期**就 abort 了。同一个 deviceType(2in1) 的 AgentKit
 *   探针现在仍返回 false —— 说明这两个 Kit 在这台设备上的可用性并不一样。
 *
 *   所以：**这个文件只应该在 canIUse 为 true 的设备上被加载。**
 *   实际使用中由 pipeline 决定注入 Real 还是 Fake 引擎。
 *
 * 它实现的是 lib/ocr.ts 里的 `OcrEngine` 接口 —— 核心算法层完全不知道
```

```
 * 这里是 Vision Kit 还是假引擎，这正是当初把 OCR 做成接口的目的。
 */
import { textRecognition } from '@kit.CoreVisionKit';
import { image } from '@kit.ImageKit';
import { common } from '@kit.AbilityKit';
import { fileIo } from '@kit.CoreFileKit';
import { OcrEngine, OcrResult } from './ocr';
/** 覆盖率估算结果。 */
interface CoverageEstimate {
  /** 识别字数 ÷ 应有字数。引擎没给几何信息时为 0。 */
  coverage: number;
  /** 参与估算的行数。 */
  lineCount: number;
  /** 识别出的（去空白后）字数。 */
  actualChars: number;
  /** 由行几何估算的应有字数。 */
  expectedChars: number;
}
/**
 * 计算图片的锐度（边缘能量的平均值），取值 0-255 量级。
 *
 * WHY THIS EXISTS — 一次失败的尝试和它的教训
 *   我最初的想法是「字符覆盖率」：用 OCR 检测到的行框算出"这一行该有多少字"，
 *   再和实际认出的字数比。**实测证明这个信号完全无效** ——
 *   最差的那张图（召回 28%）覆盖率和正常图一样（1.74 vs 1.72）。
 *
 *   失效原因很根本：expected 是从**检测到的行框**算出来的。
 *   如果 OCR 整行都没检测到，那一行的框也不存在 ——
 *   实际字数和应有字数**一起缩小**，比值自我归一化，对漏认天然免疫。
 *   **检测器的盲区，在它自己的输出里是看不见的。**
 *
 *   所以要判断"这张图是不是糊到不能用"，必须用一个
 *   **完全不依赖 OCR 输出的**参照 —— 也就是直接看像素。
 *
 * 算法：相邻像素灰度差的平均绝对值（水平 + 竖直）。
 *   图像模糊 → 边缘被抹平 → 差值变小。
 *   实现上按 step 抽样，避免对整张图逐像素求和。
 *
 * ⚠ 它是一个**相对指标**，不是标定的物理量：
 *   同一张图在不同分辨率下取值会变，所以阈值必须按设备/尺寸实测确定，
 *   不能当成通用常数。
 */
async function estimateSharpness(pm: image.PixelMap): Promise<number> {
  try {
    const info: image.ImageInfo = await pm.getImageInfo();
    const w: number = info.size.width;
    const h: number = info.size.height;
    if (w < 8 || h < 8) {
      return 0;
    }
```

```
    const buf: ArrayBuffer = new ArrayBuffer(w * h * 4);
    await pm.readPixelsToBuffer(buf);
    const px: Uint8Array = new Uint8Array(buf);
    // 抽样步长：目标约 200×100 个采样点，兼顾速度与代表性
    const stepX: number = Math.max(1, Math.floor(w / 200));
    const stepY: number = Math.max(1, Math.floor(h / 100));
    let sum: number = 0;
    let n: number = 0;
    for (let y = 0; y + stepY < h; y += stepY) {
      for (let x = 0; x + stepX < w; x += stepX) {
        const i0: number = (y * w + x) * 4;
        const i1: number = (y * w + (x + stepX)) * 4;
        const i2: number = ((y + stepY) * w + x) * 4;
        // 灰度近似：直接用 G 通道，省两次加法
        const g0: number = px[i0 + 1];
        const gx: number = Math.abs(g0 - px[i1 + 1]);
        const gy: number = Math.abs(g0 - px[i2 + 1]);
        sum += gx + gy;
        n += 2;
      }
    }
    return n > 0 ? sum / n : 0;
  } catch (e) {
    return 0;
  }
}
/** 把行检测结果换算成宽高。 */
function lineBox(line: textRecognition.TextLine): number[] {
  // 返回 [w, h]；几何信息缺失时返回 [0, 0]
  const pts: textRecognition.PixelPoint[] = line.cornerPoints;
  if (pts === undefined || pts.length < 4) {
    return [0, 0];
  }
  let minX: number = pts[0].x;
  let maxX: number = pts[0].x;
  let minY: number = pts[0].y;
  let maxY: number = pts[0].y;
  for (let i = 1; i < pts.length; i++) {
    if (pts[i].x < minX) { minX = pts[i].x; }
    if (pts[i].x > maxX) { maxX = pts[i].x; }
    if (pts[i].y < minY) { minY = pts[i].y; }
    if (pts[i].y > maxY) { maxY = pts[i].y; }
  }
  return [maxX - minX, maxY - minY];
}
/** 去掉空白后的字数。与 OcrQuality 里的归一化口径保持一致。 */
function countNonSpace(s: string): number {
  let n: number = 0;
  for (let i = 0; i < s.length; i++) {
    const c = s.charAt(i);
```

```
    if (c === ' ' || c === '\t' || c === '\n' || c === '\r' || c === '\u3000') {
      continue;
    }
    n += 1;
  }
  return n;
}
/**
 * 估算字符覆盖率。
 *
 * 原理：中日韩字形近似正方形，所以「N 个字的一行」的宽度约为 N 倍行高。
 *   应有字数 ≈ 行宽 ÷ 行高
 *   覆盖率 = 实际字数 ÷ 应有字数
 *
 * 为什么要按行算而不是按整图算：
 *   字号会在同一张图里变化（标题大、正文小），按整图算会把差异抹平。
 *   按行算，每行都用**自己的行高**做尺度，天然对字号不变。
 */
function estimateCoverage(res: textRecognition.TextRecognitionResult): CoverageEstimate {
  let actual: number = 0;
  let expected: number = 0;
  let lines: number = 0;
  const blocks: textRecognition.TextBlock[] = res.blocks;
  if (blocks !== undefined) {
    for (let i = 0; i < blocks.length; i++) {
      const ls: textRecognition.TextLine[] = blocks[i].lines;
      if (ls === undefined) {
        continue;
      }
      for (let j = 0; j < ls.length; j++) {
        const box = lineBox(ls[j]);
        const w = box[0];
        const h = box[1];
        actual += countNonSpace(ls[j].value ?? '');
        lines += 1;
        if (h > 0) {
          expected += w / h;
        }
      }
    }
  }
  const cov: CoverageEstimate = {
    coverage: expected > 0 ? actual / expected : 0,
    lineCount: lines,
    actualChars: actual,
    expectedChars: expected
  };
  return cov;
}
/**
```

```
 * 由图片锐度定置信度。
 *
 * 阈值不是拍的，是从 12 张已知答案的测试图实测出来的：
 *
 *   图              召回     锐度
 *   清晰            100%    11.6
 *   旋转 15°        100%    12.9
 *   噪声            100%    27.9
 *   旋转 6.5°       100%    12.6
 *   行楷             98%    10.1
 *   楷体            100%     7.5
 *   高斯模糊        100%     5.4
 *   低对比          100%     2.7
 *   小字号          100%     2.5
 *   强模糊           93%     2.4
 *   极低对比        100%     1.6
 *   ──────────────────────────
 *   小字+模糊        28%     1.3   ← 唯一真正失败的
 *
 * **所有识别成功的都在 1.5 以上，唯一失败的在 1.3。** 所以阈值取 1.5。
 *
 * ⚠ 必须说清楚的三个局限：
 *   1. **分隔带很窄**（1.3 vs 1.6），而样本只有 12 张合成图。
 *      这个阈值很像过拟合，换一批图很可能要重定。
 *   2. 锐度衡量的是**图片质量**，不是**识别质量**。
 *      低对比（2.7）和极低对比（1.6）看起来"糊"，但 Vision Kit 照样认出 100% ——
 *      说明 OCR 比"图片看起来清楚"要稳健得多。
 *   3. 因此**低锐度只应提示用户核对，不应自动丢弃**。
 *      界面上我们本来就不自动入库，识别结果只是填进输入框由用户确认，
 *      这个设计比任何自动判据都可靠。
 *
 * 返回值是**分档**，不是概率。
 */
function confidenceFromSharpness(sharpness: number, textLen: number): number {
  if (textLen === 0) {
    return 0;
  }
  if (sharpness <= 0) {
    // 读不到像素，无法判断 —— 给中性值，不假装有信心
    return 0.5;
  }
  if (sharpness >= 1.5) {
    return 0.9;
  }
  // 低于 1.5：实测里唯一失败的那一档。提示核对，而不是直接判死。
  return 0.3;
}
export class RealOcrEngine implements OcrEngine {
  private ctx: common.UIAbilityContext;
  constructor(ctx: common.UIAbilityContext) {
```

```
  this.ctx = ctx;
}
public name(): string {
  return 'vision-kit-ocr';
}
/**
 * 能力是否可用。
 *
 * 用 `canIUse` 而不是"试着调一下"：前者不会产生副作用，也不会因为失败而拖慢启动。
 */
public available(): boolean {
  return canIUse('SystemCapability.AI.OCR.TextRecognition');
}
/**
 * 识别一张图。
 *
 * 支持三种来源：
 *   · `rawfile:<文件名>`  —— 应用内置资源（自测用，不依赖用户数据）
 *   · `file://...`        —— 系统选择器返回的文件 URI（相册/文件）
 *   · 其他                —— 明确报错，不静默失败
 *
 * ⚠ 关于"为什么不直接调相机"
 *   调相机需要 `ohos.permission.CAMERA`，而本软件的核心主张之一是
 *   **不申请任何系统权限**（这也是"数据不出设备"在权限层面的证据）。
 *
 *   系统相册选择器（PhotoViewPicker）**不需要申请权限**：
 *   由系统界面让用户挑选，应用只拿到那一张图的 URI。
 *   所以"导入图片做识别"可以做到零权限，"调相机拍照"不行。
 *   取舍是明确的：宁可少一个入口，也不放弃零权限。
 */
public async recognize(imageUri: string): Promise<OcrResult> {
  const t0: number = Date.now();
  let file: fileIo.File | undefined = undefined;
  try {
    let src: image.ImageSource | undefined = undefined;
    if (imageUri.startsWith('rawfile:')) {
      const name: string = imageUri.substring('rawfile:'.length);
      const content: Uint8Array = await this.ctx.resourceManager.getRawFileContent(name);
      // Uint8Array 的 buffer 可能带偏移，这里拷一份干净的 ArrayBuffer
      const copy: Uint8Array = new Uint8Array(content.length);
      copy.set(content);
      src = image.createImageSource(copy.buffer as ArrayBuffer);
    } else if (imageUri.startsWith('file://') || imageUri.startsWith('datashare://')) {
      // 选择器返回的 URI：拿到 fd 再交给解码器，不必把整个文件读进内存
      file = fileIo.openSync(imageUri, fileIo.OpenMode.READ_ONLY);
      src = image.createImageSource(file.fd);
    } else {
      return {
        text: '',
        confidence: 0,
```

```
          elapsedMs: Date.now() - t0,
          error: '不支持的图片来源：' + imageUri
        };
      }
      const pm: image.PixelMap = await src.createPixelMap();
      src.release();
      // 锐度必须在 OCR **之前**算：它是独立于识别结果的参照。
      const sharp: number = await estimateSharpness(pm);
      const conf: textRecognition.TextRecognitionConfiguration = {
        isDirectionDetectionSupported: false
      };
      const res: textRecognition.TextRecognitionResult = await textRecognition.recognizeText(
        { pixelMap: pm }, conf
      );
      pm.release();
      const text: string = res.value;
      const cov = estimateCoverage(res);
      const confidence: number = confidenceFromSharpness(sharp, text.length);
      return {
        text: text,
        confidence: confidence,
        elapsedMs: Date.now() - t0,
        coverage: cov.coverage,
        lineCount: cov.lineCount,
        sharpness: sharp
      };
    } catch (e) {
      const msg: string = e instanceof Error ? e.message : '未知错误';
      return {
        text: '',
        confidence: 0,
        elapsedMs: Date.now() - t0,
        error: 'OCR 失败：' + msg
      };
    } finally {
      // 无论成功失败都要关文件描述符，否则反复识别会耗光 fd
      if (file !== undefined) {
        try {
          fileIo.closeSync(file);
        } catch (e) {
          // 关闭失败不改变识别结果，忽略
        }
      }
    }
  }
}
// ==================== pages/Index.ets ====================
/**
 * 离线资料库 —— 主界面。
 *
```

```
 * 设计原则（都来自开题答辩时老师提的批评）：
 *   1. **打开就能用**：默认进搜索页，不需要先"建库"才能看到东西
 *   2. **可解释**：每次检索都告诉用户"为什么给你这几条"（k 是怎么定的）
 *   3. **诚实降级**：OCR 不可用时明确提示改用粘贴文本，而不是静默失败
 *
 * ⚠ ArkTS 约束（L2 验证时实测踩到的，务必遵守）：
 *    · 对象字面量必须有上下文类型
 *    · 数组字面量元素类型必须可推断
 *    · 禁止 any / unknown / as unknown as
 */
import { LibraryService } from '../lib/pipeline';
import { FakeOcrEngine } from '../lib/ocr';
import { DocItem, Hit, Span, BudgetInput, SearchOptions, DocSource } from '../lib/types';
import { QueryOutcome, ImportOutcome, ImportInput } from '../lib/pipeline';
import { runSelfChecks, CheckSummary, CheckResult } from '../selfcheck/SelfCheck';
import { probeCapabilities, CapabilityProbe } from '../selfcheck/Capability';
import { common } from '@kit.AbilityKit';
import { readText, writeText, hasArchive, readTextByUri } from '../lib/librarystorage';
import { photoAccessHelper } from '@kit.MediaLibraryKit';
import { picker } from '@kit.CoreFileKit';
import { RealOcrEngine } from '../lib/realocr';
import { OcrResult } from '../lib/ocr';
import { runQualityTest, QualityRow, QUALITY_CASES } from '../selfcheck/OcrQuality';
const BUDGET: BudgetInput = { latencyBudgetMs: 500, memoryBudgetKb: 4096, batteryPercent: 80 };
// mergeAdjacent：把同一文档里相接的命中块合并成一条，避免同一段话返回多次。
const OPTS_FIXED: SearchOptions = { topK: 3, minK: 1, maxK: 20, mergeAdjacent: true };
const OPTS_AUTO: SearchOptions = { topK: 0, minK: 1, maxK: 20, mergeAdjacent: true };
@Entry
@Component
struct Index {
  // 服务实例（用假 OCR：本页要在没有 HMS 的环境里也能跑）
  private svc: LibraryService = new LibraryService(new FakeOcrEngine());
  @State currentTab: number = 0;
  @State query: string = '';
  @State hits: Hit[] = [];
  @State kUsed: number = 0;
  @State kSource: string = '-';
  @State limitedBy: string = '-';
  @State budgetReason: string = '-';
  @State candidateCount: number = 0;
  @State searchStatus: string = '输入关键词，或先在「资料」页添加一份内容';
  @State useAutoK: boolean = true;
  @State docs: DocItem[] = [];
  @State docCount: number = 0;
  @State chunkCount: number = 0;
  @State tombstone: number = 0;
  // 导入表单
  @State inputTitle: string = '';
  @State inputText: string = '';
  @State importStatus: string = '';
```

```
// 自测
@State checks: CheckResult[] = [];
@State checkSummary: string = '尚未运行';
@State checksDone: boolean = false;
@State caps: CapabilityProbe[] = [];
@State ocrStatus: string = '未运行';
@State ocrText: string = '';
@State pickStatus: string = '';
@State qualityRows: QualityRow[] = [];
@State qualityStatus: string = '未运行';
/** 持久化状态，显示在「资料」页顶部，让用户知道这次是不是真的存下来了。 */
@State persistStatus: string = '';
// ---- 详情与编辑 ----
/** 是否打开详情页。打开时主界面被替换掉，返回即关闭。 */
@State detailOpen: boolean = false;
@State detailId: string = '';
@State detailTitle: string = '';
@State detailText: string = '';
@State detailEditing: boolean = false;
@State detailStatus: string = '';
// ---- 删除确认 ----
/**
 * 待确认删除的文档 id。空串表示没有待确认的删除。
 *
 * 为什么用两步确认而不是弹窗：
 *   AlertDialog 在 2in1 的浮动窗口里位置不好控制，
 *   而"点一次变成『确认删除』、再点一次才删"是纯状态切换，
 *   不需要额外 API，行为也更容易验证。
 */
@State pendingDeleteId: string = '';
// ---- 搜索历史 ----
@State history: string[] = [];
// ---- 文件导入 ----
@State fileStatus: string = '';
aboutToAppear(): void {
  this.caps = probeCapabilities();
  this.refreshStats();
  this.restoreOrSeed();
}
// ---------------------------------------------------------------------------
// 持久化
// ---------------------------------------------------------------------------
/**
 * 从沙箱读回资料；没有存档时才放示例。
 *
 * ⚠ 这里区分两种"空"：
 *   ① 从来没有存档（首次启动）→ 放示例，让用户知道能干什么
 *   ② 有存档但里面没文档（用户删光了）→ **不放示例**，尊重用户的删除
 *   把两者混为一谈，用户体验就是"删了又回来"。
 */
```

```
private restoreOrSeed(): void {
  try {
    const ctx = getContext(this) as common.UIAbilityContext;
    if (!hasArchive(ctx)) {
      this.persistStatus = '首次启动，已放入示例资料';
      this.seedDemo();
      return;
    }
    const json: string = readText(ctx, 'library.json');
    if (json.length === 0) {
      this.persistStatus = '存档读取失败';
      return;
    }
    const skipped: number = this.svc.load(json);
    this.refreshStats();
    this.persistStatus = skipped > 0
      ? `已恢复 ${this.svc.docCount()} 份（跳过 ${skipped} 条损坏数据）`
      : `已恢复 ${this.svc.docCount()} 份资料`;
  } catch (e) {
    this.persistStatus = '恢复失败';
  }
}
/** 立即落盘。任何改动之后都要调用。 */
private persist(): void {
  try {
    const ctx = getContext(this) as common.UIAbilityContext;
    const ok = writeText(ctx, 'library.json', this.svc.save());
    if (!ok) {
      this.persistStatus = '保存失败（改动仅在本次运行有效）';
    }
  } catch (e) {
    this.persistStatus = '保存异常';
  }
}
// ---------------------------------------------------------------------------
// 数据操作
// ---------------------------------------------------------------------------
/** 首次启动放一份示例内容，避免"打开是空的、不知道能干嘛"。 */
private seedDemo(): void {
  if (this.svc.docCount() > 0) {
    return;
  }
  const demo: string = '示例资料：移动端的多智能体系统存在预算矛盾，' +
    '普通对话消耗一倍 token，而多智能体约为十五倍；手机能提供的上下文窗口只有四千到八千。' +
    '因此取舍是强制的：注入哪些工具、消息用什么形式、这一步在哪里执行，都要在预算内决定。';
  this.svc.importDocument({ title: '示例：预算调度笔记', source: 'note', uri: 'demo://1', text: demo })
    .then((_r: ImportOutcome) => {
      this.refreshStats();
      // 示例也要落盘：否则下次启动又会判定"没有存档"，再塞一遍
      this.persist();
```

```
    });
}
private refreshStats(): void {
  this.docs = this.svc.documents();
  this.docCount = this.svc.docCount();
  this.chunkCount = this.svc.chunkCount();
  this.tombstone = this.svc.tombstoneCount();
}
private doSearch(): void {
  const q: string = this.query.trim();
  if (q.length === 0) {
    this.searchStatus = '请先输入关键词';
    return;
  }
  if (this.svc.chunkCount() === 0) {
    this.searchStatus = '资料库还是空的，请先在「资料」页添加内容';
    return;
  }
  const opts: SearchOptions = this.useAutoK ? OPTS_AUTO : OPTS_FIXED;
  const out: QueryOutcome = this.svc.query(q, BUDGET, opts);
  this.hits = out.hits;
  this.kUsed = out.k;
  this.kSource = out.kSource;
  this.limitedBy = out.limitedBy;
  this.budgetReason = out.budgetReason;
  this.candidateCount = out.diagnostics.candidateCount;
  this.searchStatus = out.hits.length === 0
    ? '没有找到相关内容'
    : `找到 ${out.hits.length} 条（候选 ${out.diagnostics.candidateCount} 条）`;
  this.pushHistory(q);
}
private doImport(): void {
  const t: string = this.inputText.trim();
  if (t.length === 0) {
    this.importStatus = '内容不能为空';
    return;
  }
  const title: string = this.inputTitle.trim().length > 0 ? this.inputTitle.trim() : '未命名资料';
  const src: DocSource = 'note';
  this.svc.importDocument({ title: title, source: src, uri: 'note://' + Date.now().toString(), text: t })
    .then((r: ImportOutcome) => {
      this.importStatus = r.message;
      if (r.ok) {
        this.inputTitle = '';
        this.inputText = '';
        this.refreshStats();
        // 每次改动立即落盘：HarmonyOS 里 UIAbility 可能被系统直接回收，
        // "退出时保存"不可靠。
        this.persist();
      }
```

```
    })
    .catch((_e: Error) => { this.importStatus = '导入失败'; });
}
private doRemove(id: string): void {
  this.svc.removeDocument(id);
  this.refreshStats();
  this.hits = [];
  this.searchStatus = '已删除，请重新搜索';
  this.persist();
}
// ---------------------------------------------------------------------------
// 详情与编辑
// ---------------------------------------------------------------------------
private openDetail(id: string): void {
  for (let i = 0; i < this.docs.length; i++) {
    if (this.docs[i].id === id) {
      this.detailId = id;
      this.detailTitle = this.docs[i].title;
      this.detailText = this.docs[i].text;
      this.detailEditing = false;
      this.detailStatus = '';
      this.detailOpen = true;
      return;
    }
  }
}
/** 保存编辑。走 replaceDocument，保持文档在列表中的位置。 */
private saveDetail(): void {
  const t = this.detailText;
  if (t.trim().length === 0) {
    this.detailStatus = '内容不能为空';
    return;
  }
  const input: ImportInput = {
    title: this.detailTitle.trim().length > 0 ? this.detailTitle.trim() : '未命名资料',
    source: 'note',
    uri: 'note://edited/' + this.detailId,
    text: t
  };
  this.svc.replaceDocument(this.detailId, input)
    .then((r: ImportOutcome) => {
      this.detailStatus = r.message;
      if (r.ok) {
        this.detailEditing = false;
        this.refreshStats();
        this.persist();
        // 内容变了，旧结果已失效
        this.hits = [];
        this.searchStatus = '资料已更新，请重新搜索';
      }
```

```
    })
    .catch((_e: Error) => { this.detailStatus = '保存失败'; });
}
// ---------------------------------------------------------------------------
// 删除确认（两步）
// ---------------------------------------------------------------------------
private askDelete(id: string): void {
  this.pendingDeleteId = id;
}
private confirmDelete(): void {
  const id = this.pendingDeleteId;
  this.pendingDeleteId = '';
  if (id.length === 0) {
    return;
  }
  this.doRemove(id);
}
private cancelDelete(): void {
  this.pendingDeleteId = '';
}
// ---------------------------------------------------------------------------
// 搜索历史
// ---------------------------------------------------------------------------
/** 记一条历史。同一条移到最前，最多保留 8 条。 */
private pushHistory(q: string): void {
  const out: string[] = [];
  out.push(q);
  for (let i = 0; i < this.history.length && out.length < 8; i++) {
    if (this.history[i] !== q) {
      out.push(this.history[i]);
    }
  }
  this.history = out;
}
private clearHistory(): void {
  this.history = [];
}
// ---------------------------------------------------------------------------
// 文件导入
// ---------------------------------------------------------------------------
/**
 * 用系统文件选择器导入 txt / md。
 *
 * DocumentViewPicker 和相册选择器一样**不需要申请权限**：
 * 由系统界面让用户挑文件，应用只拿到那一个文件的 URI。
 * 所以零权限的主张在这里同样成立。
 */
private pickFileAndImport(): void {
  this.fileStatus = '请选择 txt 或 md 文件…';
  try {
```

```
    const ctx = getContext(this) as common.UIAbilityContext;
    // 局部变量不能叫 picker：会遮蔽导入的模块名（编译报「used before its declaration」）
    const filePicker: picker.DocumentViewPicker = new picker.DocumentViewPicker(ctx);
    const opts: picker.DocumentSelectOptions = new picker.DocumentSelectOptions();
    opts.maxSelectNumber = 1;
    const filters: string[] = ['.txt', '.md', '.markdown'];
    opts.fileSuffixFilters = filters;
    filePicker.select(opts)
      .then((uris: string[]) => {
        if (uris.length === 0) {
          this.fileStatus = '没有选择文件';
          return;
        }
        this.importFromUri(ctx, uris[0]);
      })
      .catch((_e: Error) => { this.fileStatus = '打开文件选择器失败'; });
  } catch (e) {
    this.fileStatus = '无法打开文件选择器';
  }
}
private importFromUri(ctx: common.UIAbilityContext, uri: string): void {
  try {
    const text = readTextByUri(uri);
    if (text.length === 0) {
      this.fileStatus = '文件为空或无法读取';
      return;
    }
    let name: string = uri;
    const slash = uri.lastIndexOf('/');
    if (slash >= 0 && slash + 1 < uri.length) {
      name = uri.substring(slash + 1);
    }
    this.svc.importDocument({ title: name, source: 'file', uri: uri, text: text })
      .then((r: ImportOutcome) => {
        this.fileStatus = r.message;
        if (r.ok) {
          this.refreshStats();
          this.persist();
        }
      })
      .catch((_e: Error) => { this.fileStatus = '导入失败'; });
  } catch (e) {
    this.fileStatus = '读取文件失败';
  }
}
private runChecks(): void {
  this.checkSummary = '运行中…';
  runSelfChecks()
    .then((s: CheckSummary) => {
      this.checks = s.results;
```

```
      this.checksDone = true;
      this.checkSummary = s.failed === 0
        ? `全部通过：${s.passed} / ${s.total}`
        : `有失败：${s.failed} / ${s.total}`;
    })
    .catch((_e: Error) => { this.checkSummary = '自测运行失败'; });
}
/**
 * 从相册选一张图，识别后把文字填进导入框。
 *
 * 为什么用选择器而不是调相机：
 * 相机需要 ohos.permission.CAMERA，会破坏本软件"零权限"的主张；
 * 系统相册选择器不需要任何权限，由系统界面让用户挑选，
 * 应用只拿到那一张图的 URI。
 */
private pickImageAndOcr(): void {
  this.pickStatus = '请在系统相册中选择图片…';
  try {
    const ctx = getContext(this) as common.UIAbilityContext;
    // 局部变量不叫 picker：会遮蔽从 @kit.CoreFileKit 导入的 picker 模块
    const photoPicker: photoAccessHelper.PhotoViewPicker = new photoAccessHelper.PhotoViewPicker();
    const opts = new photoAccessHelper.PhotoSelectOptions();
    opts.MIMEType = photoAccessHelper.PhotoViewMIMETypes.IMAGE_TYPE;
    opts.maxSelectNumber = 1;
    photoPicker.select(opts)
      .then((res: photoAccessHelper.PhotoSelectResult) => {
        if (res.photoUris.length === 0) {
          this.pickStatus = '没有选择图片';
          return;
        }
        const uri = res.photoUris[0];
        this.pickStatus = '识别中…';
        const engine = new RealOcrEngine(ctx);
        engine.recognize(uri)
          .then((r: OcrResult) => {
            if (r.error !== undefined) {
              this.pickStatus = r.error;
              return;
            }
            // 识别结果填进导入框，由用户确认后再入库 ——
            // 不直接入库，因为识别可能有错，用户需要看一眼
            this.inputText = r.text;
            if (this.inputTitle.length === 0) {
              this.inputTitle = '图片导入 ' + new Date().toLocaleDateString();
            }
            this.ocrText = r.text;
            this.ocrStatus = `识别成功：${r.text.length} 字 · 用时 ${r.elapsedMs} ms`;
            this.pickStatus = '已填入下方输入框，确认后点「加入资料库」';
          })
          .catch((e: Error) => { this.pickStatus = '识别失败：' + e.message; });
```

```
      })
      .catch((e: Error) => { this.pickStatus = '选择图片失败：' + e.message; });
  } catch (e) {
    this.pickStatus = '无法打开相册选择器';
  }
}
/** OCR 质量测试：用已知答案的合成图集量化退化。 */
private runQuality(): void {
  this.qualityStatus = '运行中…';
  this.qualityRows = [];
  try {
    const ctx = getContext(this) as common.UIAbilityContext;
    const engine = new RealOcrEngine(ctx);
    if (!engine.available()) {
      this.qualityStatus = '该设备不支持 OCR，无法测试';
      return;
    }
    runQualityTest(engine)
      .then((rows: QualityRow[]) => {
        this.qualityRows = rows;
        let ok = 0;
        for (let i = 0; i < rows.length; i++) {
          if (rows[i].error === '') { ok += 1; }
        }
        this.qualityStatus = `完成：${ok} / ${rows.length} 张有结果`;
      })
      .catch((e: Error) => { this.qualityStatus = '测试失败：' + e.message; });
  } catch (e) {
    this.qualityStatus = '初始化失败';
  }
}
/** L4：真实 OCR 实测（内置测试图，不依赖相机）。 */
private runOcrTest(): void {
  this.ocrStatus = '识别中…';
  this.ocrText = '';
  try {
    // getContext(this) 在页面里拿到的就是 UIAbilityContext
    const ctx = getContext(this) as common.UIAbilityContext;
    const engine = new RealOcrEngine(ctx);
    if (!engine.available()) {
      this.ocrStatus = '该设备不支持 OCR';
      return;
    }
    engine.recognize('rawfile:ocr_sample.png')
      .then((r: OcrResult) => {
        if (r.error !== undefined) {
          this.ocrStatus = r.error;
          return;
        }
        this.ocrStatus = `识别成功：${r.text.length} 字 · 用时 ${r.elapsedMs} ms`;
```

```
        this.ocrText = r.text;
      })
      .catch((e: Error) => { this.ocrStatus = 'OCR 调用失败：' + e.message; });
  } catch (e) {
    this.ocrStatus = 'OCR 初始化失败';
  }
}
// ---------------------------------------------------------------------
// 视图片段
// ---------------------------------------------------------------------
/** 把带高亮的文本渲染成多个 Span。 */
@Builder
highlightText(text: string, spans: Span[]) {
  Text() {
    Span(text.substring(0, spans.length > 0 ? spans[0].start : text.length))
    ForEach(spans, (s: Span) => {
      Span(text.substring(s.start, s.end))
        .fontColor($r('app.color.danger'))
        .fontWeight(FontWeight.Bold)
    }, (_s: Span, i: number) => i.toString())
    if (spans.length > 0) {
      Span(text.substring(spans[spans.length - 1].end))
    }
  }
  .fontSize(13)
  .width('100%')
}
@Builder
searchTab() {
  Column({ space: 10 }) {
    Row({ space: 8 }) {
      TextInput({ placeholder: '搜索你的资料…', text: this.query })
        .layoutWeight(1)
        .height(42)
        .fontSize(14)
        .onChange((v: string) => { this.query = v; })
        .onSubmit(() => { this.doSearch(); })
      Button('搜索')
        .height(42)
        .fontSize(14)
        .onClick(() => { this.doSearch(); })
    }
    .width('100%')
    Row({ space: 8 }) {
      Text('检索深度').fontSize(12).fontColor($r('app.color.text_secondary'))
      Toggle({ type: ToggleType.Switch, isOn: this.useAutoK })
        .onChange((on: boolean) => { this.useAutoK = on; })
      Text(this.useAutoK ? '按得分分布自适应' : '固定 3 条')
        .fontSize(12)
        .fontColor($r('app.color.accent'))
```

```
}
.width('100%')
// 搜索历史：点一下就用那个词重搜
if (this.history.length > 0) {
  Row({ space: 6 }) {
    Text('最近').fontSize(10).fontColor($r('app.color.text_hint'))
    ForEach(this.history, (h: string) => {
      Text(h)
        .fontSize(10.5)
        .fontColor($r('app.color.accent'))
        .padding({ left: 8, right: 8, top: 3, bottom: 3 })
        .backgroundColor($r('app.color.accent_bg'))
        .borderRadius(10)
        .onClick(() => {
          this.query = h;
          this.doSearch();
        })
    }, (h: string, i: number) => h + i.toString())
    Text('清空')
      .fontSize(10).fontColor($r('app.color.text_hint'))
      .onClick(() => { this.clearHistory(); })
  }
  .width('100%')
}
// 决策解释条
if (this.kUsed > 0) {
  Column({ space: 3 }) {
    Text(`返回 ${this.kUsed} 条 · 来源 ${this.kSource} · 限制方 ${this.limitedBy}`)
      .fontSize(11).fontColor($r('app.color.accent')).width('100%')
    Text(`候选 ${this.candidateCount} 条 · 资源约束：${this.budgetReason}`)
      .fontSize(10).fontColor($r('app.color.text_tertiary')).width('100%')
  }
  .padding(8)
  .width('100%')
  .backgroundColor($r('app.color.info_bg'))
  .borderRadius(6)
}
Text(this.searchStatus).fontSize(11).fontColor($r('app.color.text_tertiary')).width('100%')
List({ space: 8 }) {
  ForEach(this.hits, (h: Hit) => {
    ListItem() {
      Column({ space: 4 }) {
        Row({ space: 6 }) {
          Text(this.titleOf(h.chunk.docId)).fontSize(13).fontWeight(FontWeight.Medium).layoutWeight(1)
          Text(h.score.toFixed(2)).fontSize(10).fontColor($r('app.color.accent'))
        }
        .width('100%')
        this.highlightText(h.chunk.text, h.spans)
      }
      .padding(10)
```

```
          .width('100%')
          .backgroundColor($r('app.color.card_bg'))
          .borderRadius(8)
        }
      }, (h: Hit) => h.chunk.id)
    }
    .layoutWeight(1)
    .width('100%')
  }
  .padding(14)
  .width('100%')
  .height('100%')
  .backgroundColor($r('app.color.page_bg'))
}
private titleOf(docId: string): string {
  for (let i = 0; i < this.docs.length; i++) {
    if (this.docs[i].id === docId) {
      return this.docs[i].title;
    }
  }
  return docId;
}
@Builder
libraryTab() {
  Column({ space: 10 }) {
    Row({ space: 12 }) {
      Text(`资料 ${this.docCount} 份`).fontSize(12).fontColor($r('app.color.text_primary'))
      Text(`片段 ${this.chunkCount} 个`).fontSize(12).fontColor($r('app.color.text_primary'))
      if (this.tombstone > 0) {
        Text(`待整理 ${this.tombstone}`).fontSize(12).fontColor($r('app.color.danger'))
      }
    }
    .width('100%')
    // 持久化状态：让用户看得见"这次有没有存下来"
    if (this.persistStatus.length > 0) {
      Row({ space: 6 }) {
        Text('存档').fontSize(10).fontColor($r('app.color.ok'))
        Text(this.persistStatus).fontSize(10).fontColor($r('app.color.text_secondary')).layoutWeight(1)
      }
      .width('100%')
    }
    Column({ space: 8 }) {
      TextInput({ placeholder: '标题（可留空）', text: this.inputTitle })
        .height(40).fontSize(13)
        .onChange((v: string) => { this.inputTitle = v; })
      TextArea({ placeholder: '粘贴或输入要入库的内容…', text: this.inputText })
        .height(110).fontSize(13)
        .onChange((v: string) => { this.inputText = v; })
      Row({ space: 8 }) {
        Button('从相册选图识别')
```

```
      .height(34).fontSize(12)
      .backgroundColor($r('app.color.accent'))
      .onClick(() => { this.pickImageAndOcr(); })
    Button('导入文件')
      .height(34).fontSize(12)
      .backgroundColor($r('app.color.accent_bg')).fontColor($r('app.color.accent'))
      .onClick(() => { this.pickFileAndImport(); })
  }
  .width('100%')
  if (this.pickStatus.length > 0) {
    Text(this.pickStatus).fontSize(10).fontColor($r('app.color.text_secondary')).width('100%')
  }
  if (this.fileStatus.length > 0) {
    Text(this.fileStatus).fontSize(10).fontColor($r('app.color.accent')).width('100%')
  }
  Row({ space: 8 }) {
    Button('加入资料库')
      .height(38).fontSize(13)
      .onClick(() => { this.doImport(); })
    if (this.importStatus.length > 0) {
      Text(this.importStatus).fontSize(11).fontColor($r('app.color.ok')).layoutWeight(1)
    }
  }
  .width('100%')
}
.padding(10)
.width('100%')
.backgroundColor($r('app.color.card_bg'))
.borderRadius(8)
List({ space: 6 }) {
  ForEach(this.docs, (d: DocItem) => {
    ListItem() {
      Row({ space: 8 }) {
        Column({ space: 2 }) {
          Text(d.title).fontSize(13).width('100%')
          Text(`${d.source} · ${d.text.length} 字 · 点开查看`)
            .fontSize(10).fontColor($r('app.color.text_tertiary')).width('100%')
        }
        .layoutWeight(1)
        .alignItems(HorizontalAlign.Start)
        // 删除两步确认：一点就删在手机上太容易误触，
        // 而资料删掉是找不回来的。
        if (this.pendingDeleteId === d.id) {
          Row({ space: 6 }) {
            Button('确认删除')
              .fontSize(11).height(28)
              .backgroundColor($r('app.color.danger')).fontColor(Color.White)
              .onClick(() => { this.confirmDelete(); })
            Button('取消')
              .fontSize(11).height(28)
```

```
                .backgroundColor($r('app.color.subtle_bg')).fontColor($r('app.color.text_primary'))
                .onClick(() => { this.cancelDelete(); })
            }
          } else {
            Button('删除')
              .fontSize(11).height(28)
              .backgroundColor($r('app.color.subtle_bg')).fontColor($r('app.color.danger'))
              .onClick(() => { this.askDelete(d.id); })
          }
        }
        .padding(10)
        .width('100%')
        .backgroundColor($r('app.color.card_bg'))
        .borderRadius(8)
        .onClick(() => { this.openDetail(d.id); })
      }
    }, (d: DocItem) => d.id)
  }
  .layoutWeight(1)
  .width('100%')
 }
 .padding(14)
 .width('100%')
 .height('100%')
 .backgroundColor($r('app.color.page_bg'))
}
@Builder
advancedTab() {
 Column({ space: 10 }) {
  // 能力探测（canIUse，零风险：不 import 任何 Kit）
  Column({ space: 5 }) {
    Text('设备能力探测（canIUse）').fontSize(14).fontWeight(FontWeight.Bold).width('100%')
    Text('只用 canIUse 查询，不 import 任何 Kit —— 因此不可能把应用拖崩')
      .fontSize(10).fontColor($r('app.color.text_tertiary')).width('100%')
    ForEach(this.caps, (c: CapabilityProbe) => {
      Row({ space: 8 }) {
        Text(c.available ? '✓' : '✗')
          .fontSize(13).width(16)
          .fontColor(c.available ? $r('app.color.ok') : $r('app.color.danger'))
        Column({ space: 1 }) {
          Text(c.label).fontSize(11.5).width('100%')
          Text(c.syscap.replace('SystemCapability.', '')).fontSize(9)
            .fontColor($r('app.color.text_hint')).width('100%')
        }
        .layoutWeight(1)
        .alignItems(HorizontalAlign.Start)
      }
      .width('100%')
      .padding({ left: 8, right: 8, top: 4, bottom: 4 })
    }, (c: CapabilityProbe) => c.syscap)
```

```
      }
      .padding(12)
      .width('100%')
      .backgroundColor($r('app.color.card_bg'))
      .borderRadius(8)
      // OCR 质量测试（已知答案，量化退化）
      Column({ space: 6 }) {
        Text('OCR 质量测试（' + QUALITY_CASES.length.toString() + ' 张已知答案的图）
').fontSize(14).fontWeight(FontWeight.Bold).width('100%')
        Text('同一段文字，不同退化条件 —— 算字符级召回/精确率，而不是靠印象')
          .fontSize(10).fontColor($r('app.color.text_tertiary')).width('100%')
        Row({ space: 10 }) {
          Button('运行质量测试').height(36).fontSize(13)
            .onClick(() => { this.runQuality(); })
          Text(this.qualityStatus).fontSize(11).fontColor($r('app.color.text_secondary')).layoutWeight(1)
        }
        .width('100%')
        if (this.qualityRows.length > 0) {
          Row({ space: 3 }) {
            Text('图').fontSize(9).fontColor($r('app.color.text_hint')).width(86)
            Text('召回').fontSize(9).fontColor($r('app.color.text_hint')).width(36)
            Text('精确').fontSize(9).fontColor($r('app.color.text_hint')).width(36)
            Text('F1').fontSize(9).fontColor($r('app.color.text_hint')).width(32)
            Text('覆盖').fontSize(9).fontColor($r('app.color.text_faint')).width(36)
            Text('锐度').fontSize(9).fontColor($r('app.color.accent')).width(38)
            Text('置信').fontSize(9).fontColor($r('app.color.accent')).width(34)
            Text('ms').fontSize(9).fontColor($r('app.color.text_hint')).width(30)
          }
          .width('100%')
          Row({ space: 3 }) {
            Text('覆盖=旧方案(已验证无效)  锐度=新方案  置信由锐度给出')
              .fontSize(8).fontColor($r('app.color.text_faint')).width('100%')
          }
          .width('100%')
          ForEach(this.qualityRows, (r: QualityRow) => {
            Row({ space: 3 }) {
              Text(r.label).fontSize(9.5).width(86)
              Text(r.error === '' ? r.recall.toFixed(0) + '%' : '—').fontSize(9.5)
                .fontColor(r.recall >= 90 ? $r('app.color.ok') : (r.recall >= 60 ? $r('app.color.warn') :
$r('app.color.danger'))).width(36)
              Text(r.error === '' ? r.precision.toFixed(0) + '%' : '—').fontSize(9.5).width(36)
              Text(r.error === '' ? r.f1.toFixed(0) + '%' : '—').fontSize(9.5).width(32)
              Text(r.coverage > 0 ? r.coverage.toFixed(2) : '—').fontSize(9.5)
                .fontColor($r('app.color.text_faint')).width(36)
              Text(r.sharpness > 0 ? r.sharpness.toFixed(1) : '—').fontSize(9.5)
                .fontColor($r('app.color.accent')).width(38)
              Text(r.confidence > 0 ? r.confidence.toFixed(2) : '—').fontSize(9.5)
                .fontColor(r.confidence >= 0.9 ? $r('app.color.ok')
                  : (r.confidence >= 0.7 ? $r('app.color.warn') : $r('app.color.danger'))).width(34)
              Text(r.error === '' ? r.ms.toString() : '—').fontSize(9.5)
                .fontColor($r('app.color.text_tertiary')).width(30)
            }
```

```
      .width('100%')
      .padding({ top: 2, bottom: 2 })
    }, (r: QualityRow, i: number) => r.label + i.toString())
  }
}
.padding(12)
.width('100%')
.backgroundColor($r('app.color.card_bg'))
.borderRadius(8)
// L4：真实 OCR 实测
Column({ space: 6 }) {
  Text('OCR 实测（CoreVisionKit）').fontSize(14).fontWeight(FontWeight.Bold).width('100%')
  Text('用内置测试图跑一次真实 OCR —— 不依赖相机')
    .fontSize(10).fontColor($r('app.color.text_tertiary')).width('100%')
  Row({ space: 10 }) {
    Button('运行 OCR 实测')
      .height(36).fontSize(13)
      .onClick(() => { this.runOcrTest(); })
    Text(this.ocrStatus).fontSize(11).fontColor($r('app.color.text_secondary')).layoutWeight(1)
  }
  .width('100%')
  if (this.ocrText.length > 0) {
    Text(this.ocrText)
      .fontSize(11)
      .width('100%')
      .padding(8)
      .backgroundColor($r('app.color.page_bg'))
      .borderRadius(6)
  }
}
.padding(12)
.width('100%')
.backgroundColor($r('app.color.card_bg'))
.borderRadius(8)
Column({ space: 6 }) {
  Text('四个环节，四个信号').fontSize(14).fontWeight(FontWeight.Bold).width('100%')
  Text('这是本项目的研究主张：调度不能用单一的「剩余预算」驱动。')
    .fontSize(11).fontColor($r('app.color.text_secondary')).width('100%')
  Text('· 感知（OCR）→ 识别置信度决定收下/重拍/丢弃')
    .fontSize(11).width('100%')
  Text('· 索引 → 内容规模决定分块大小')
    .fontSize(11).width('100%')
  Text('· 检索 → 得分分布决定返回几条')
    .fontSize(11).width('100%')
  Text('· 资源 → 延迟/内存/电量决定还取得起几条')
    .fontSize(11).width('100%')
}
.padding(12)
.width('100%')
.backgroundColor($r('app.color.card_bg'))
```

```
    .borderRadius(8)
    Column({ space: 8 }) {
      Row({ space: 8 }) {
        Button('运行设备内自测')
          .height(38).fontSize(13)
          .onClick(() => { this.runChecks(); })
        Text(this.checkSummary).fontSize(12)
          .fontColor(this.checkSummary.indexOf('全部通过') >= 0 ? $r('app.color.ok') : $r('app.color.text_secondary'))
          .layoutWeight(1)
      }
      .width('100%')
      if (this.checksDone) {
        List({ space: 1 }) {
          ForEach(this.checks, (c: CheckResult) => {
            ListItem() {
              Row({ space: 8 }) {
                Text(c.passed ? '✓' : '✗')
                  .fontSize(13).width(18)
                  .fontColor(c.passed ? $r('app.color.ok') : $r('app.color.danger'))
                Text(c.name).fontSize(11).layoutWeight(1)
                if (!c.passed) {
                  Text(c.detail).fontSize(10).fontColor($r('app.color.danger'))
                }
              }
              .padding({ left: 8, right: 8, top: 5, bottom: 5 })
              .width('100%')
            }
          }, (c: CheckResult, i: number) => c.name + i.toString())
        }
        .layoutWeight(1)
        .width('100%')
      }
    }
    .padding(12)
    .width('100%')
    .layoutWeight(1)
    .backgroundColor($r('app.color.card_bg'))
    .borderRadius(8)
  }
  .padding(14)
  .width('100%')
  .height('100%')
  .backgroundColor($r('app.color.page_bg'))
}
/** 详情页：查看与编辑一份资料。打开时替换主界面，返回即关闭。 */
@Builder
detailView() {
  Column({ space: 0 }) {
    // 顶栏
    Row({ space: 10 }) {
```

```
  Button('← 返回')
    .fontSize(13).height(34)
    .backgroundColor($r('app.color.subtle_bg')).fontColor($r('app.color.text_primary'))
    .onClick(() => {
      this.detailOpen = false;
      this.detailEditing = false;
      this.detailStatus = '';
    })
  Text(this.detailEditing ? '编辑资料' : '资料详情')
    .fontSize(14).fontWeight(FontWeight.Bold).layoutWeight(1)
  if (!this.detailEditing) {
    Button('编辑')
      .fontSize(12).height(34)
      .onClick(() => {
        this.detailEditing = true;
        this.detailStatus = '';
      })
  }
}
.width('100%')
.padding({ left: 14, right: 14, top: 12, bottom: 10 })
.backgroundColor($r('app.color.page_bg'))
Column({ space: 8 }) {
  if (this.detailEditing) {
    TextInput({ placeholder: '标题', text: this.detailTitle })
      .height(40).fontSize(13)
      .onChange((v: string) => { this.detailTitle = v; })
    TextArea({ placeholder: '正文', text: this.detailText })
      .layoutWeight(1).fontSize(12.5)
      .onChange((v: string) => { this.detailText = v; })
  } else {
    Text(this.detailTitle)
      .fontSize(15).fontWeight(FontWeight.Bold).width('100%')
    Text(`${this.detailText.length} 字`)
      .fontSize(10).fontColor($r('app.color.text_tertiary')).width('100%')
    Scroll() {
      Text(this.detailText)
        .fontSize(12.5)
        .width('100%')
        .lineHeight(21)
    }
    .layoutWeight(1)
    .width('100%')
    .align(Alignment.TopStart)
  }
}
.layoutWeight(1)
.padding(14)
.width('100%')
// 底栏
```

```
    Row({ space: 10 }) {
      if (this.detailEditing) {
        Button('保存')
          .height(38).fontSize(13).layoutWeight(1)
          .onClick(() => { this.saveDetail(); })
        Button('取消')
          .height(38).fontSize(13).layoutWeight(1)
          .backgroundColor($r('app.color.subtle_bg')).fontColor($r('app.color.text_primary'))
          .onClick(() => {
            this.detailEditing = false;
            // 放弃修改：重新从列表里读回原值
            this.openDetail(this.detailId);
            this.detailStatus = '';
          })
      } else {
        Text(this.detailStatus.length > 0 ? this.detailStatus : '只读。点「编辑」可修改内容。')
          .fontSize(11).fontColor($r('app.color.text_tertiary')).layoutWeight(1)
      }
    }
    .width('100%')
    .padding({ left: 14, right: 14, top: 10, bottom: 14 })
    .backgroundColor($r('app.color.page_bg'))
  }
  .width('100%')
  .height('100%')
  .backgroundColor($r('app.color.page_bg'))
}
build() {
  // 外层只负责铺底与「居中」，内容被限制最大宽度。
  //
  // WHY A MAX WIDTH INSTEAD OF A TWO-PANE LAYOUT
  //   平板横屏（约 1280×800vp）和 2in1 上，如果不限宽，
  //   一行正文会被拉到 1200vp 以上 —— 眼睛要横扫一整屏，非常难读。
  //   限宽 + 居中是最小改动、且对三种形态都成立的做法：
  //     手机竖屏（约 400vp） → 占满，无变化
  //     手机横屏（约 800vp） → 占满，无变化
  //     平板/2in1 横屏       → 居中在 900vp，两侧留白
  //   真正的双栏（列表 + 详情同屏）是更好的形态，但改动面大，留作后续；
  //   限宽已经解决了"拉伸难看"这个主要问题。
  Column() {
    Column() {
      if (this.detailOpen) {
        this.detailView()
      } else {
        Tabs({ barPosition: BarPosition.End, index: this.currentTab }) {
          TabContent() { this.searchTab() }.tabBar('搜索')
          TabContent() { this.libraryTab() }.tabBar('资料')
          TabContent() { this.advancedTab() }.tabBar('高级')
        }
        .onChange((i: number) => { this.currentTab = i; })
```

```
          .barHeight(52)
          .width('100%')
          .height('100%')
        }
      }
      .width('100%')
      .height('100%')
      .constraintSize({ maxWidth: 900 })
    }
    .width('100%')
    .height('100%')
    .alignItems(HorizontalAlign.Center)
    .backgroundColor($r('app.color.page_bg'))
  }
}
// ==================== entryability/EntryAbility.ets ====================
import { AbilityConstant, UIAbility, Want } from '@kit.AbilityKit';
import { hilog } from '@kit.PerformanceAnalysisKit';
import { window } from '@kit.ArkUI';
export default class EntryAbility extends UIAbility {
  onCreate(want: Want, launchParam: AbilityConstant.LaunchParam): void {
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onCreate');
  }
  onDestroy(): void {
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onDestroy');
  }
  onWindowStageCreate(windowStage: window.WindowStage): void {
    // Main window is created, set main page for this ability
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onWindowStageCreate');
    windowStage.loadContent('pages/Index', (err) => {
      if (err.code) {
        hilog.error(0x0000, 'testTag', 'Failed to load the content. Cause: %{public}s', JSON.stringify(err) ?? '');
        return;
      }
      hilog.info(0x0000, 'testTag', 'Succeeded in loading the content.');
    });
  }
  onWindowStageDestroy(): void {
    // Main window is destroyed, release UI related resources
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onWindowStageDestroy');
  }
  onForeground(): void {
    // Ability has brought to foreground
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onForeground');
  }
  onBackground(): void {
    // Ability has back to background
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onBackground');
  }
}
```